From Agents to Institutions: A Field Study of Institutional Control in an AI-Staffed Prediction-Market Desk

Paper type: Technical Report / Empirical Systems Study

Author: Wes Zheng

Project repository: https://github.com/wes-zheng/ai_institutions

Abstract

Modern agent systems increasingly support tool use, handoffs, memory, tracing, durable execution, guardrails, and multi-agent orchestration. These primitives improve agent capability, but they do not by themselves answer a different question: can AI workers produce accountable, authorized, reviewable, stoppable, and improvable work under operational pressure?

This paper reports a field study of an AI-staffed prediction-market desk under human-owner governance boundaries. The desk is used as a high-skill stress test rather than as a trading contribution. The domain forces the organization to handle uncertain evidence, changing external state, risk limits, no-action decisions, execution authority, delayed outcome truth, and post-outcome learning.

We use **institutional control** to mean the durable mechanisms by which an AI organization assigns ownership, constrains authority, records evidence, verifies outputs, validates closure, handles no-action, reconciles outcomes, and mutates doctrine. The central empirical finding is that many failures that appear to be agent failures are better diagnosed as organizational failures: missing ownership, broad intent not compiled into executable work, tool access mistaken for authority, plausible artifacts accepted without verifier state, stale messages treated as current work, completion confused with closure, and learning trapped in chat or tickets rather than becoming durable doctrine.

A key finding is that the institutional layer did not appear fully formed: repeated failures were promoted into durable role, work-record, message, verifier, replay, tool, and playbook artifacts under manager or owner authority.

The study contributes a framework for AI organizations as a level above agents and workflows, bounded field evidence from one human-owned deployment, and organization-level metrics for evaluating AI labor systems. It does not claim general causal certainty. It studies the institutional infrastructure for auditable, controllable, and improvable AI labor rather than deployment-wide performance improvement.

1. Introduction

Agentic AI systems are usually evaluated through the behavior of a model, an agent, a workflow, or a team of agents. Those levels matter. A useful worker must be able to reason, use tools, plan, interact with external systems, and hand off work. But human labor is not performed by isolated intelligence. It is performed inside institutions that define who owns work, who has authority, what evidence is accepted, when work may stop, who verifies it, how closure is recorded, and how the organization changes after failure.

This paper argues that the path from agents to AI labor runs through institutions.

The distinction is practical. A capable agent can produce a plausible analysis while no one owns the next step. A tool-using agent can see market or account information without being authorized to act on it. A reviewer agent can provide helpful commentary without making an acceptance decision. A workflow can finish while the governing record remains unclosed. A memory system can store lessons while the next employee still acts from stale doctrine. These are not just prompt-engineering problems. They are organizational-control problems.

We study those problems through an AI-staffed prediction-market desk. The deployment was AI-staffed for daily operational labor and human-owned for goals, risk boundaries, institutional design, and publication judgment. It was composed of AI employees with durable roles, messages, work records, playbooks, review gates, execution boundaries, manager verification, replay records, and doctrine changes. The empirical domain is deliberately demanding. Weather and climate prediction-market work involves uncertainty modeling, source

freshness, market interpretation, risk review, execution discipline, no-action judgment, and delayed settlement or outcome reconciliation. It also leaves artifacts when the organization fails.

The paper is not about a trading strategy. It is about what had to be built around AI workers before their work became accountable, reviewable, stoppable, and improvable.

1.1 From Agent Capability To Auditable Labor Control

Agent-centered systems ask whether an AI can complete a task. Organization-centered systems ask whether the work can be owned, authorized, verified, stopped, closed, replayed, and adapted.

Agent-centric question	Organization-centric question
Can the agent act?	Who owns the work now?
Can the agent call tools?	What does tool evidence authorize?
Can agents hand off?	Did accountability transfer to the right owner?
Can a workflow finish?	Was closure valid and accepted?
Can memory persist?	Did learning become durable doctrine?
Can a reviewer critique output?	Did a verifier make a stateful accept/repair/reroute/decline decision?

The empirical system started with familiar agent ingredients: specialized roles, tools, work tracking, messages, and instructions. Over time, failures forced a more institutional design. Broad tasks became child work records. Personas became role contracts. Comments became message-trigger rules. Critique became verifier state. Completion became manager closure. Retros became replay and doctrine-mutation paths. The workboard became a state machine that compiled broad intent into live umbrellas, retro umbrellas, child paths, activations, gates, and replay obligations. Work records became the source of truth rather than conversation. Playbooks became operating doctrine rather than optional documentation. No-action became a valid output. Replay became the bridge from outcome truth to organizational learning.

1.2 Thesis

The thesis is:

> This study suggests that high-skill AI labor takes more than agent capability: it needs institutional-control mechanisms that make ownership, authority, evidence, verification, no-action, closure, replay, and governed adaptation auditable and controllable.

This thesis does not imply that models, tools, memory, or workflow graphs are unimportant. It implies that they are insufficient units of analysis for high-skill labor. The same model and tool stack can behave very differently depending on the institution around it.

The organizing frame is four-part: institutional control converts agent outputs into auditable labor by governing state, authority, communication, and learning.

Pillar	Main mechanisms	What failure it exposes or controls
State control	work records, workboard, closure	chat-as-state and fake completion
Authority control	role contracts, verifier gates, tool boundaries	capability mistaken for permission
Communication control	messages, inbox staleness, no-reply	false motion and stale triggers
Learning control	replay, doctrine mutation, case-only classification	lessons trapped in memory or tickets

1.3 Research Questions

This study asks five questions:

RQ1. Organizational primitives: What primitives are needed to turn agent-like workers into AI employees inside a durable organization?

RQ2. Auditability and controllability: Which organizational primitives make ownership, authority, evidence, closure, and learning failures visible, bounded, or controllable?

RQ3. Decision work: Can an AI organization produce verifier-reconstructible technical artifacts rather than plausible prose?

RQ4. Learning: How does delayed outcome truth become durable organizational change rather than one-off correction?

RQ5. Generalization: Which design principles transfer beyond prediction-market trading to other high-skill technical organizations?

1.4 Contributions

This paper makes three contributions.

1. **Conceptual contribution:** It defines AI organization and institutional control as an evaluation level above models, agents, workflows, and teams.
2. **Empirical contribution:** It reports field evidence from a human-owned, AI-staffed high-skill operational desk showing ownership, authority, evidence, workboard-state, topology, closure, no-action, replay, and mutation failures.
3. **Evaluation contribution:** It proposes organization-level metrics and sanitized artifact field sets for assessing AI labor systems without reducing evaluation to task completion or model capability.

2. Related Work And Positioning

The public baseline as of April 2026 is strong. Agent frameworks have matured rapidly. Tool protocols have become standard infrastructure. Benchmarks increasingly evaluate long-horizon web, computer-use, software-engineering, and tool-use tasks. Multi-agent systems have explored role decomposition, group chat, social simulation, and standard operating procedures.

This paper does not claim those directions are wrong. It claims they operate mostly below the organizational layer.

These systems improve execution. This paper studies institutional acceptance, authority, closure, no-action, replay, and doctrine mutation.

2.1 Agent SDKs And Execution Infrastructure

OpenAI's Agents SDK and related agent tooling provide primitives for agents, tools, handoffs, guardrails, tracing, sandboxed execution, skills, and durable execution. OpenAI's April 2026 Agents SDK update describes a model-native harness, tool use through MCP, sandboxed execution, progressive disclosure through skills, and checkpoint-like restoration of agent state after environment loss [OpenAI, 2026a]. The Agents SDK documentation describes tracing of model calls, tool calls, handoffs, and guardrails [OpenAI, 2026c], and guardrails around inputs, outputs, and custom function-tool invocations rather than a complete institutional authority model for all tool use [OpenAI, 2026b].

These are valuable execution primitives. They help developers run, observe, interrupt, and constrain agent workflows. They do not by themselves define whether a worker has authority to close a work item, whether a no-action outcome is valid, whether an old message is superseded, whether a verifier must accept or reroute an artifact, or where reusable learning must land.

2.2 Tool Protocols And MCP

The Model Context Protocol standardizes tool and context integration for AI applications. The current MCP specification defines a JSON-RPC protocol with hosts, clients, servers, resources, prompts, tools, sampling,

progress, cancellation, logging, and security considerations [Model Context Protocol, 2025b]. Related 2025-11-25 specification material emphasizes schema conformance and protocol metadata [Model Context Protocol, 2025a]. Recent MCP production-pattern work, published as a preprint, argues that production deployments still need additional mechanisms such as identity propagation, adaptive tool/timeout budgeting, and structured error semantics [Srinivasan, 2026]. This report addresses a separate layer above tool integration: institutional authority, work ownership, verifier state, no-action, replay, and doctrine mutation.

MCP is a major infrastructure layer, but a protocol for tool access is not an organization. It does not, by itself, decide that an execution employee may only act inside a risk-authorized envelope, that a reviewer can decline but not execute, that a manager alone may mutate durable doctrine, or that a message is stale because a governing work record has moved on. This paper positions organization-protocol semantics above MCP: employee identity, role authority, work state, doctrine, message lifecycle, verification, replay, and institutional mutation.

2.3 Durable Workflow And Multi-Agent Frameworks

LangGraph provides durable execution and persistence for long-running workflows, including human-in-the-loop and interruption recovery [LangChain, 2026]. CrewAI combines agents, crews, flows, tasks, guardrails, callbacks, triggers, persistence, and human-in-the-loop patterns [CrewAI, 2026]. AutoGen and related systems explore multi-agent conversations, teams, handoffs, and event-driven workflows [Wu et al., 2023]. MetaGPT encodes standard operating procedures into prompt sequences and role-based agent collaboration for software work [Hong et al., 2024].

These systems move beyond a single agent, but the primary abstraction remains workflow, team, or collaboration. An AI organization adds stronger semantics: not only that an agent handed off, but whether accountability moved to the correct owner; not only that a workflow resumed, but whether the governing work record is truthful; not only that a role exists, but whether the role owns or is forbidden from a decision; not only that an SOP was available, but whether reusable learning changed durable doctrine without losing prior constraints.

2.4 Social Simulation And Agent Memory

Generative Agents demonstrated believable social behavior by storing natural-language experiences, synthesizing reflections, and retrieving memories to plan behavior in a simulated town [Park et al., 2023]. That line of work is important for humanlike interaction and social simulation. This paper takes a different stance: humanlike behavior is not the goal. Accountable institutional labor is the goal. A worker can be socially plausible while still failing to own, authorize, verify, close, or learn correctly.

2.5 Agent Benchmarks And Evaluation

Agent benchmarks increasingly evaluate more realistic tasks. WebArena provides realistic web environments and long-horizon web tasks, reporting large gaps between agents and humans in early systems [Zhou et al., 2024]. OSWorld evaluates computer-use agents in real desktop environments [Xie et al., 2024]. SWE-bench evaluates whether language models can resolve real GitHub issues [Jimenez et al., 2024]. Tau-bench evaluates tool-agent-user interaction in domains with policies and tools, emphasizing reliability over repeated trials [Yao et al., 2024]. AgentBench evaluates LLMs as agents across multiple interactive environments [Liu et al., 2024], and TheAgentCompany moves closer to workplace labor by evaluating agents in a simulated software-company environment [Xu et al., 2024]. A survey of LLM-agent evaluation, submitted in 2025 and revised in 2026, identifies planning, tool use, memory, application benchmarks, generalist agents, cost-efficiency, safety, robustness, and scalable fine-grained evaluation as major themes [Yehudai et al., 2026].

These benchmarks strengthen the shift from static task evaluation toward agentic and workplace task performance. This report asks a different question: whether AI labor is owned, authorized, closed, stopped, replayed, and mutated as institutional state. An agent can pass a task benchmark while still failing organizationally; conversely, an organization can correctly stop a task that a benchmark might treat as incomplete.

2.6 Business Process Management And Process Mining

Business process management and process-mining systems model tasks, events, handoffs, conformance, and process improvement [Dumas et al., 2018, van der Aalst, 2016]. This report is related, but the control problem

differs because probabilistic AI workers require explicit role authority, evidence demotion, message freshness, valid no-action, verifier acceptance, and replay-to-doctrine mutation around model calls.

2.7 Institutions, Norms, And Agent Governance

The paper also sits in a longer multi-agent systems tradition. Normative multi-agent systems study how norms, roles, enforcement, and compliance shape autonomous-agent interaction [Boella et al., 2006]. Electronic institutions make conventions explicit for open multi-agent systems and use institutional artifacts to shape participation and compliance [Esteva et al., 2004].

Recent LLM-agent governance preprints and specifications are closer still. Institutional AI reframes multi-agent alignment from preference engineering in agent-space to mechanism design in institution-space, using governance graphs, public manifests, legal states, transitions, sanctions, restorative paths, and audit logs [Syrnikov et al., 2026, Pierucci et al., 2026]. Agent Control Protocol frames agent action as a temporal admission-control problem between agent intent and system state mutation, emphasizing identity, capability scope, delegation, risk, execution history, and auditability [Fernandez, 2026, Agent Control Protocol, 2026].

This paper is complementary. It does not propose a formal admission-control protocol or a governance graph. It brings the institutional lens to empirical AI labor: work records, messages, role authority, verifier topology, no-action, closure, replay, and doctrine mutation in a live operational desk.

2.8 Organizational Learning And High-Reliability Work

Organizational-learning and high-reliability work study how institutions convert operational experience into procedures, roles, checks, and routines [Argyris and Schön, 1978, Weick and Sutcliffe, 2015]. This report adapts that lens to AI labor: replay records, role/playbook versions, doctrine patches, verifier gates, tool-parity repairs, and activation messages are treated as AI-native organizational-learning artifacts.

We use organizational-learning and high-reliability work as positioning lenses, not as evidence that this deployment achieved high-reliability-organization status. The mechanism claim is narrower: durable learning must land in institutional artifacts rather than remaining in private memory, one-off reflection, chat, or ticket commentary.

2.9 Positioning Summary

Public baseline	What it provides	Organization-level gap addressed here
Agent SDKs	tools, handoffs, guardrails, tracing, sandboxing	durable authority, closure, doctrine, no-action, replay
MCP	standardized tool/resource/prompt access	employee identity, work ownership, role authority, verifier state
Workflow graphs	persistence, routing, interrupts	organizational truth, manager finality, institutional mutation
BPM and process mining	task/event models, handoffs, conformance, process improvement	AI-specific authority, evidence demotion, no-action, and replay-to-doctrine mutation
Multi-agent teams	collaboration and role decomposition	authority contracts, stateful handoffs, accepted closure
SOP agents	procedural decomposition	versioned doctrine, targeted mutation, recurrence checks
Agent benchmarks	task success in environments	organization-level auditability and learning
Institutional AI / ACP	formal governance graphs, admission control, auditability	empirical labor records, closure, no-action, replay, doctrine mutation

Public baseline	What it provides	Organization-level gap addressed here
Organizational learning and high-reliability work	experience-to-routine learning and resilience practices	machine-executable replay, versioned doctrine, and activation-gated mutation

The paper's claim is not that agent systems lack infrastructure. The claim is that high-skill AI labor may need an additional institutional layer.

3. Framework: AI Organization And Institutional Control

3.1 Level Hierarchy

The proposed hierarchy is:

Level	Primary question	Limitation for auditable labor control
Model	Can it reason or generate?	no durable ownership or authority
Agent	Can it act with tools?	weak institutional accountability
Workflow	Can steps be routed?	weak doctrine, authority, and learning semantics
Team	Can specialists coordinate?	conversation can masquerade as state
Organization	Can work be owned, authorized, verified, stopped, replayed, and adapted?	target level

Figure 1. From Models To AI Organizations.

```
model
  -> agent
  -> workflow / team
  -> organization

unit of control:
generation -> tool action -> routed steps -> institutional labor
```

Takeaway: the paper studies the organizational layer where ownership, authority, closure, replay, and mutation become evaluable.

An organization is not just "many agents." It is a durable control system around workers.

3.2 Definition

An **AI organization** is a durable socio-technical control system in which AI employees perform bounded work under explicit role contracts, authority limits, governing work records, reusable doctrine, verifier gates, and outcome-driven learning loops.

"AI employee" is used here as an operational term for a persistent institutional principal with role, authority, inbox, tools, and history. It is not a legal, moral, or anthropomorphic claim.

This definition deliberately separates worker capability from organizational controllability. The organization is responsible for converting probabilistic model outputs into accountable labor artifacts.

3.3 Institutional Control

Institutional control is the durable mechanism set by which an AI organization assigns ownership, constrains authority, records evidence, verifies outputs, validates closure, handles no-action, reconciles outcomes, and mutates doctrine. In this operational sense, it rests on six negations:

1. The model is not the system of record.
2. Conversation is not durable organizational state.
3. Tool access is not authority.
4. Completion is not self-declared.
5. Memory is not doctrine.
6. Reflection is not learning until it changes the right durable layer or is explicitly classified as case-only.

Persistence is treated as substrate, not as a standalone novelty. The relevant institutional state is employee identity, role text, playbooks, inboxes, governing work records, tool authority, and replay obligations. Private memory may help continuity, but it is a hint, not the system of record. When private memory conflicts with the governing record, role text, or playbook doctrine, the durable institutional surface controls.

Institutional control is therefore less about making a single worker more autonomous and more about making a group of workers governable.

3.4 Organizational Primitives

Primitive	Function	Failure mode exposed or controlled
Employee principal	Persistent worker identity, inbox, role, tools, status	stateless task execution
Role contract	Owned decisions, forbidden decisions, outputs, escalation	persona drift and unauthorized action
Governing work record	Case truth: owner, status, evidence, blocker, next action	chat-as-state and fake progress
Workboard state machine	Intent compiled into umbrellas, child records, gate states, owners, activations, closure, and replay	vague intent that never becomes executable labor
Work message	Trigger and handoff with exact ask and reply expectation	missed activation and acknowledgment loops
Playbook doctrine	Reusable workflow and evidence rules	repeated one-off correction
Evidence surface	Accepted source/tool outputs attached to records	unverifiable reasoning
Verifier gate	Accept, repair, reroute, decline, block, or approve	plausible but incomplete artifacts
No-action state	Valid stop, decline, stand-down, already-done, explicit zero	forced action and false completion
Replay record	Outcome reconciliation and learning classification	learning detached from truth
Institutional mutation	Changes to roles, playbooks, tools, staffing, topology, workboards, or messages	repeated failure risk, unactivated fixes, and learning trapped in local context

Figure 2. Institutional Control Stack.

```
human-owner boundary
  -> organization topology and manager finality
  -> employee principal + role contract
  -> playbook doctrine
  -> governing work record
```

```
-> message / inbox trigger
-> evidence packet + verifier gate
-> closure / no-action / replay / mutation
```

Takeaway: the model invocation is only one worker cycle inside a larger control stack.

3.5 Propositions

The empirical study is organized around five propositions.

Proposition 1. Ownership: AI work becomes more auditable and controllable when every substantive work item has a governing record with a current owner and exact next action.

Proposition 2. Authority: AI work is more controllable when role authority is separated from tool access and downstream action depends on explicit authorization.

Proposition 3. Evidence: AI decision artifacts become more reviewable when evidence, assumptions, and decision structure are stored outside private model context.

Proposition 4. No-action: AI organizations benefit from first-class states for repair, decline, blocked, stand-down, already-done, no-send, and explicit zero.

Proposition 5. Replay: Outcome reconciliation becomes more traceable when replay is linked to the role, playbook, evidence, and work-state artifacts active at the time of action.

4. Empirical Setting And Study Design

4.1 Empirical Setting

The empirical setting is an AI-staffed prediction-market desk working in weather and climate markets. The organization performed research, evidence collection, market-path selection, risk review, execution or no-action decisions, manager closure, and replay. It was AI-staffed in daily operational labor and human-owned at the level of goals, risk boundaries, institutional design, and publication judgment.

The domain is useful because it stresses several properties common to high-skill technical work:

- external evidence changes over time;
- intermediate artifacts must be verifiable by another role;
- action can be unsupported even when analysis is plausible;
- stopping can be correct;
- delayed outcome truth can evaluate earlier decisions;
- organizational mistakes leave durable traces in work records, messages, reviews, and replay.

The desk is not presented as evidence that AI systems should trade, nor as financial advice. It is a field setting where institutional failures are observable.

4.1.1 Pre-Institutional Reference Condition

The reference condition was not a separate model architecture or controlled benchmark baseline. It was the earlier operating mode of the same deployment: role-like agents using tools, work tracking, messages, and instructions before mature ownership, authority, closure, no-action, replay, and mutation semantics were introduced.

The study therefore compares observed failures in that pre-institutional reference condition with bounded mature-state audits after institutional-control mechanisms were introduced. It should not be read as a randomized comparison or as a benchmark-style model evaluation.

4.2 Unit Of Analysis

The primary empirical unit is a **market-path episode**: a bounded path from intake through research, review, action or no-action, closure, and replay where applicable.

Episodes may end in several valid states:

- no-send or no-action;
- repair request;
- risk decline;
- execution stand-down;
- executed action followed by replay;
- manager closure;
- durable doctrine or role/tool/topology mutation;
- case-only learning classification.

This matters because the organization is not evaluated only on actions taken. It is evaluated on whether it reached the truthful state.

4.3 Data Artifacts

The empirical record consists of:

- governing work records;
- task and handoff messages;
- role contracts and role versions;
- playbooks and playbook versions;
- verifier decisions;
- manager closure records;
- replay and retro records;
- owner-directed change records;
- tool/profile/tool-set records;
- realized outcome records where available.

This report uses sanitized extraction summaries, not raw internal records. A first evidence extraction pass deep-read selected work records and comment threads, performed mechanism keyword sweeps, and produced sanitized evidence packets. A message evidence pass sampled manager, execution-lead, and verifier inboxes for no-reply suppression, self-trigger discipline, stale/unauthorized reroute, and verifier decision patterns. A bounded sprint-window pass extracted denominator-labeled counts for selected Results metrics.

4.4 Study Scope And Evidence Corpus

This is a field study and mechanism paper. It does not claim general causal certainty. It identifies recurring organizational failure modes and the control mechanisms that made them observable, bounded, or repairable.

The deployment context is summarized below. The full corpus is not used as a denominator in this report; bounded counts are reported only for selected windows with enough sanitized evidence to code the relevant fields.

Deployment attribute	Value
Full deployment period	Multi-week deployment; deep audit windows cover Apr 25-27, 2026, with exploratory spot-check records from earlier periods
AI employee roles	Multiple persistent AI employee principals across the role families below; full roster count is not reported
Active role families	Qualitatively includes management, research, verification, execution, replay, doctrine, and tooling functions
Work records in full corpus	More than 2,000 internal work records exist; not used as a corpus-wide denominator here
Messages in full corpus	Full count not estimated; `M1` requested 260 inbox rows for mechanism evidence

Deployment attribute	Value
Playbook versions	Versioned doctrine exists; count-level estimate is not reported
Models used	LLM API models; exact provider/configuration is not disclosed in this report and not used as causal evidence
Human-owner interventions	Bounded S1 sample includes one owner directive; full-corpus intervention count is not estimated

The bounded empirical counts in this report come from selected windows and message samples.

The selected windows are bounded mature-state audits, not a corpus-wide performance estimate of the entire deployment.

Artifact type	Count used in this report	Window / source	Used for
Market-path live child records	46	S1, R1, V1 selected sprint windows, Apr 25-27 2026	ownership, verifier, no-action, replay linkage
Work records in selected windows	80	live children, replay children, and live/retro umbrellas	state control, closure, replay, parent/child truth
S1 final closeout records	20	S1 final manager verification sample	runtime-contract compliance, closure validity, no-message closeout
S1 no-execution paths	6	S1 final live dispositions	no-action quality and no-replay rationale
S1 ordered/replay paths	14	S1 replay children	replay coverage and delayed outcome truth
Replay child records across S1/R1/V1	28	selected sprint/retro windows	replay learning and no-replay separation
Message rows requested	260	manager, execution-lead, and verifier inbox sample	no-reply, stale/reroute, self-trigger, verifier-decision examples
Message evidence packets	6	high-signal message extraction	message-trigger discipline and loop suppression
S1 functional verifier approvals	at least 16	S1 functional-verifier comments	verifier topology and approval behavior
S1 repair-required verifier decisions	at least 10	S1 functional-verifier comments	evidence sufficiency and repair routing
S1 observed reroutes	4	S1 comments	stateful verifier/manager correction
S1 changed-envelope approvals	3	S1 comments	verifier decisions that changed downstream action
S1 human-owner directive	1	S1 umbrella open directive	human-owner boundary and setup role
Durable mutation artifacts	qualitative mechanism evidence only	selected evolution/tool records	supports mechanism description, not aggregate mutation-rate claims

Artifact type	Count used in this report	Window / source	Used for
P1 pre-institutional/reference spot-check	30	earlier work-record metadata sample	selected-window bias check and reference contrast
B1 broader non-S1/R1/V1 spot-check	75	shallow metadata sample across other periods/statuses	mechanism-presence check outside selected mature windows

The study does not isolate model-version effects. Because model identity and configuration are not used as causal evidence, all claims are limited to institutional auditability and mechanism tracing rather than model-independent performance improvement.

4.5 Sample Windows

The Results section uses three named sprint windows:

- **S1:** Apr 26 20-location live sprint plus retro. This is the primary bounded sample: 20 live child records, 14 ordered/replay paths, 6 no-execution/no-replay paths, and 20 final manager-reviewed live dispositions.
- **R1:** Apr 25 daily-high live sprint plus retro. This is replay and parent-closeout augmentation: 20 live child records and 9 replay children.
- **V1:** Apr 27 mini sprint plus retro. This is a compact recency check: 6 live child records and 5 replay children.

The message evidence sample `M1` requested 260 inbox rows across manager, execution-lead, and verifier inboxes and produced 6 sanitized mechanism packets. It is not a full inbox census.

4.6 Sampling Rationale

The selected windows were chosen because they represent mature-state operation after the relevant institutional-control mechanisms were active and because they contained complete live, closeout, and replay artifacts. They are not claimed to be representative of the full deployment. Their purpose is to test whether the mature control stack made ownership, authority, no-action, closure, and replay auditable in bounded operational episodes.

Episodes were excluded from bounded metrics when the work path lacked enough sanitized evidence to code owner, state, verifier decision, or replay status. This exclusion rule supports cleaner denominator-labeled audits, but it also means the sample should not be read as a full deployment rate estimate.

4.7 Broader Spot-Check Against Selection Bias

To reduce the risk that the paper only reports clean mature-state windows, we added a shallow broader spot-check. `P1` was selected from the earliest available substantive weather/work-record search results in late-March and early-April records, excluding administrative-only records when obvious. `B1` was selected from non-S1/R1/V1 records across multiple earlier operational periods and statuses, including live child, replay, duplicate/canceled, done, todo, and backlog records where available. These are not random samples from a formal database export; they are availability-bias checks against the selected mature windows.

Records were coded from work-record metadata and description snippets only. A field counted as clear only when the record exposed the relevant owner, next action or blocker, closure state, verifier state, replay/no-replay state, or no-action state without needing private comment-thread context. These codes measure state visibility, not decision correctness.

Sample	Records	Owner present	Next clear	Closure clear	Verifier clear	Replay clear	No-action clear
`P1` reference spot-check	30	21/30	18/30	22/30	10/18; N/A 12	8/14; N/A 16	6/13; N/A 17
`S1` mature deep audit	20	20/20	20/20	20/20	20/20	20/20	6/6; N/A 14
`B1` broader spot-check	75	62/75	51/75	58/75	34/55; N/A 20	30/46; N/A 29	12/26; N/A 49

For S1, verifier clear refers to manager-reviewed final dispositions; replay clear means each path had replay or explicit no-replay classification. The spot-check supports two limited claims. First, S1 is cleaner than the earlier reference sample on fields the institutional-control stack was designed to make explicit. Second, institutional-control semantics appear outside S1/R1/V1, but not with enough coverage to infer full-corpus rates. The selected windows should therefore be read as mature-state deep audits, with P1/B1 as exploratory context.

4.8 Intervention Timeline

The organization evolved through a sequence of intervention families. The important point is not exact calendar chronology; it is the failure pressure that caused each durable layer to emerge. The organization did not become more controlled through one design insight. Each institutional layer appeared after concrete failure pressure made the previous agent/team abstraction insufficient.

The timeline should be read as institutional growth from failure pressure into durable artifact forms. Role forms answered persona drift. Work-record fields answered chat-as-state. Message rules answered missed activation and false motion. Verifier forms answered plausible-but-unauthorized artifacts. Replay forms answered learning trapped in tickets or private context.

Stage	Failure pressure	Intervention	Changed durable layer	Evidence status
T0	Useful specialist agents lacked stable ownership and closure	Persistent AI employees	employee principal, role, inbox	source-backed mechanism evidence
T1	Roles blurred recommendation, approval, execution, closure, and doctrine editing	Role contracts	role text and role families	extracted
T2	Lessons stayed in tickets, messages, or memory	Playbooks as doctrine	playbook versions	extracted
T3	Conversation appeared to own truth	Governing work records	workboard state	extracted
T4	Assignment or comments did not reliably wake employees	Direct work messages	message protocol	extracted

Stage	Failure pressure	Intervention	Changed durable layer	Evidence status
T5	Employees could act on stale messages	Inbox pre-check and suppression	runtime contract, inbox tools, messages	extracted
T6	AI workers could self-certify plausible artifacts	Verifier topology	roles, playbooks, gates	extracted
T7	Complete work remained open or review-ready without closure	Manager verification queue	workboard, manager role, runtime contract	extracted
T8	Agents over-produced action	No-action states	review gates and work states	extracted
T9	Live work stayed fake-active while awaiting delayed outcome truth	Replay split	workboard and replay records	extracted
T10	Reflection remained commentary	Replay-to-mutation loop	mutation destinations	extracted
T11	Long doctrine rewrites risked compression and drift	Targeted doctrine patches	role and playbook tools	mechanism evidence; bounded counts not reported
T12	Capability and authority were confused	Dual tool plane and tool parity	tool registry and role authority	extracted
T13	More employees increased routing ambiguity	Pods and role families	organization topology	extracted
T14	Fixes landed in the wrong layer or were not activated	Adaptive institution loop	institutional mutation process	extracted

Replay-to-mutation destinations can include roles, playbooks, tools, staffing, topology, workboards, or messages; the table uses the compact label `mutation destinations`.

The same history can be summarized as a 0-to-1 evolution arc from weak agent-team forms to institutional-control forms:

Layer	Early form	Mature institutional form
Roles	specialist personas	authority-bounded job contracts and role-family interfaces
Workboard	task tracker	institutional state machine
Messages	chat or notification	auditable work-transfer trigger
Playbooks	optional docs	versioned runtime doctrine
Tools/MCP	capability surface	person-role-aware authority surface
Harness	task prompt	common employee runtime contract
Organization	many agents	routable topology with finality and adaptation

The intervention logic is summarized as a before/after mechanism table. The "after" column describes the

mature institutional mechanism observed in bounded samples or mechanism evidence; it does not claim corpus-wide performance improvement.

Failure pressure	Before institutional control	After institutional control	Evidence type	What this does not prove
Completion confused with closure	Work could appear done while still open or review-ready.	Manager closeout required a final state or named dependency.	S1 closeout audit	Does not prove a global closure-validity rate.
Review as commentary	Reviewer feedback could refine prose without creating an authority state.	Verifiers made accept, repair, reroute, decline, or review decisions.	S1 verifier comments	Does not establish all reviews got better.
Message false motion	Replies and self-triggers could repeat without changing work truth.	No-reply and stale-message suppression became valid outcomes.	M1 packets and S1 closeouts	Does not prove loop elimination.
Broad intent without executable frontier	A high-level sprint request could leave routing, owner, gate, and replay obligations implicit.	The workboard compiled intent into live umbrella, retro umbrella, child paths, direct activations, gate states, closeout, and replay.	S1/R1/V1 sprint structure and workboard evolution	Does not prove this topology is optimal.
Delayed outcome truth	Learning could stay in chat, tickets, or worker context.	Replay classified closure, outcome truth, and learning destination.	S1 replay records	Does not prove recurrence reduction.
Tool access mistaken for authority	Tool visibility or proposal generation could imply actionability.	Execution required role authority and verifier gates.	Mechanism cases	Does not establish complete execution control.

4.9 Evidence Claim Labels

To avoid overclaiming, each result is labeled by evidence type: bounded metric claim, mechanism evidence claim, conceptual claim, design implication, or defined but not estimated in this study. Appendix G lists the allowed wording and evidence basis for major claims.

4.10 Coding And Evidence Extraction

Evidence was extracted from work records, message logs, verifier comments, manager closeouts, replay records, and doctrine changes. Each artifact was coded, when available, for:

- current owner;
- current state;
- exact next action;
- evidence basis;
- role authority;
- verifier decision type;
- no-action state;
- replay state;
- mutation destination;

- human-owner intervention;
- limitation or missing denominator.

Claims in the Results section are reported only when supported by bounded denominators or sanitized mechanism packets. Global organization-wide rates require a full corpus extraction and are not claimed in this report.

Each sanitized evidence packet was required to identify the claim, failure mode, intervention, changed durable layer, observed post-change behavior or recurrence check, sanitized artifact, and limitation. This packaging rule was used to keep empirical claims mechanism-grounded rather than asking readers to trust a narrative of improvement.

4.11 Analysis Strategy

The analysis combines:

1. **Mechanism tracing:** identifying failure pressure, intervention, changed durable layer, and post-change behavior.
2. **Artifact audit:** checking whether work records, messages, role contracts, playbooks, reviews, closure records, and replay records support the claim.
3. **Case studies:** writing bounded cases where the mechanism is visible.
4. **Bounded metric extraction:** defining organization-level metrics and extracting sample-labeled denominators where the records support them.
5. **Limitations accounting:** recording negative cases, missing recurrence windows, and human-owner boundaries.

5. Method: Institutional Control Stack

This section describes the method as a stack of organizational mechanisms. Each mechanism emerged because a simpler agent/team/workflow abstraction failed to control a real operational failure mode.

5.1 Common Employee Runtime Contract

The organization used a **common employee runtime contract** for employee cycles. It is not a prompt contribution in the narrow sense. It is a runtime contract that makes each model invocation answer organizational questions before work can move:

```
employee cycle =
  trigger
  + authority
  + doctrine
  + governing record
  + one authorized action
  + audit outcome
```

The contract frames the model as an employee, not as the system of record. A substantive cycle starts from an inbound message, identifies the governing work record, checks authority and evidence, reads relevant doctrine, verifies whether the trigger is still actionable, takes one authorized next step, and leaves a fixed planning/execution stamp with exactly one primary outcome.

A worker cycle was also bounded against backlog sweeping: one trigger had to end in one authorized outcome, not opportunistic cleanup of adjacent work.

The key runtime move is an authority lattice plus evidence demotion. The employee does not treat all retrieved text as instruction. Role text, playbooks, work records, messages, comments, tool outputs, and copied material answer different institutional questions.

Truth-layer separation: role text owns authority, playbooks own reusable procedure, work records own case truth, messages own activation, and replay owns outcome learning.

Layer	Owns	Does not own
Role text	Employee-specific authority, scope, owned decisions, forbidden decisions, required outputs, and role-specific escalation	Reusable workflow doctrine or current case truth
Playbooks	Reusable doctrine, gates, decision criteria, source standards, review standards, and replay standards	Employee authority or ticket-specific state
Governing work records	Current case truth: owner, state, evidence, blocker, latest decision, next owner, next action, closure, and replay status	Reusable doctrine or role authority
Messages, comments, tool outputs, retrieved content	Evidence, triggers, observations, and handoff requests	Durable institutional authority, role changes, or reusable doctrine

Contract semantic	Organizational meaning	Failure controlled
Authority lattice	Sources are ranked by institutional authority, not recency.	Newer messages, comments, or tool outputs override durable doctrine.
Evidence demotion	Retrieved, quoted, or tool-produced text is evidence unless adopted by higher authority.	Instruction-like evidence silently changes role or workflow.
Durable instruction layers	Role, playbook, and work record own different kinds of truth.	Case facts leak into doctrine, or doctrine remains trapped in tickets.
Governing work record	Every substantive cycle anchors to persistent case truth.	Conversation becomes the system of record.
Inbound-message trigger	Work transfer depends on explicit activation.	Assignment fields or mentions silently fail to wake employees.
Inbox staleness gate	The trigger is classified as actionable, superseded, or already done.	Duplicate replies, stale writes, and state-sync loops.
Planning/execution split	No substantive action before work, authority, state, doctrine, and next step are known.	Execution begins before authority or playbook review.
Explicit handoff and silence	Handoffs name owner/action/reply expectation; no-message is valid when no action remains.	Awareness is mistaken for transfer; acknowledgments create false motion.
Manager-only mutation	Durable people and doctrine changes require manager authority.	Tool access becomes institutional authority.
Fixed stamps and primary outcomes	The governing record receives planning fields, execution fields, and one outcome class.	Agent traces exist, but organizational state remains unauditable.

The paper treats this contract as a portable organization-protocol concept. It can be implemented with different tools, workboards, and model stacks. Its core contribution is semantic: in this deployment, worker cycles were more auditable and controllable when constrained by authority, state, doctrine, trigger discipline, review, no-action, replay, mutation, and audit rules.

The runtime contract let the organization operate with incomplete doctrine without collapsing into chat. Each cycle still had to locate governing state, respect authority order, demote messages and tool outputs to evidence unless adopted by higher authority, take one authorized next step, and leave audit state. As repeated failures exposed missing structure, managers and owners could promote fixes into role contracts, playbooks, work-

record fields, message rules, verifier gates, replay doctrine, or tool surfaces. This is evidence of institutional form maturation in one deployment, not proof of general autonomous self-improvement.

5.2 Employee Principals

An AI employee is a persistent principal, not a transient prompt. It has an identity, role, title, status, inbox, tool set, and history. This made it possible to ask: who was asked to act, what role governed the action, what messages did the worker receive, and what authority did the worker hold?

The novelty is not naming agents like employees. The novelty is treating identity as institutional state. An employee can be hired, updated, given or denied tools, messaged, reviewed, and terminated. Those are organizational state transitions, not prompt decorations.

5.3 Role Contracts

Roles evolved from humanlike job descriptions into authority contracts. A role contract specifies:

- employee-specific mandate;
- owned decisions;
- forbidden decisions;
- required outputs;
- handoff boundaries;
- escalation rules;
- non-goals.

This separated expertise from authority. A research employee could produce an evidence packet without approving execution. A verifier could accept or decline an artifact without placing an order. An executor could act only inside an authorized envelope. A manager could close, reroute, or mutate doctrine, while ordinary employees could only propose durable changes.

A role family also defines an interface: normal upstream artifact, owned transformation, downstream output, and escalation boundary, so hiring or backfilling preserves handoff contracts rather than only adding another capable model.

The general principle is: **tool capability and subject expertise do not grant institutional authority.**

5.4 Governing Work Records

Every substantive cycle required a governing work record: the persistent artifact that owns case-specific truth. The record holds current owner, state, evidence, blocker, next action, review status, replay linkage, and closure basis.

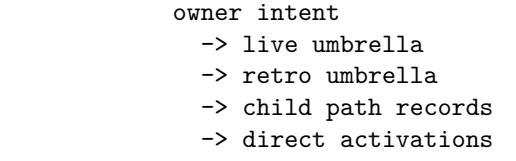
This addressed a recurring problem: conversation could make work feel complete while the durable work state remained ambiguous. The governing record became the organization's answer to "what is true now?"

Work records also made parent/child truth explicit. A parent sprint or retro could not be closed merely because some direct children looked complete if a deeper live descendant still held the real blocker or next action. Mature workboard doctrine required recursive descendant truth unless a structural severance was recorded.

5.5 Workboard As Institutional State Machine

The workboard became more than a tracker. It was the institutional state machine that turned broad owner intent into executable AI labor. A sprint launch was a compilation step.

Figure 3. Workboard As Institutional State Machine.



```
-> verifier gates
-> close / no-action / replay
```

Takeaway: broad owner intent becomes routable, verifiable, closeable work only after it is compiled into durable workboard state.

The live umbrella owned the active frontier. The retro umbrella owned later outcome truth and learning obligations. Child records owned episode truth: current owner, gate state, evidence, blocker, next action, and closure or no-action basis. Direct messages activated one employee for one child path rather than turning a broad sprint request into a general inbox sweep.

This made the workboard a frontier controller. A parent record could not truthfully close while a child still governed live work. A parent also should not remain active after the child frontier was truthfully complete. Replay obligations moved to the retro surface so live records did not stay fake-active while waiting for delayed outcome truth.

The portable design claim is that high-skill AI labor benefits from a state substrate that compiles vague intent into small, routable, verifiable work objects with explicit activation and closure semantics.

5.6 Work Messages

Messages became work triggers rather than chat. A valid work message names the work item, exact ask, exact next action, and whether reply is needed.

This design emerged because tracker assignment, comments, and mentions were insufficient to wake AI employees reliably. It later evolved further because messages themselves can become stale or duplicate. The mature protocol added inbox pre-checks, `ACTIONABLE`, `SUPERSEDED`, and `ALREADY_DONE` classifications, no-reply suppression, and limits on self-messages.

Inbox transparency was scoped. Employees checked newer relevant messages for the same work item to classify the trigger as actionable, superseded, or already done. They were not meant to use inbox access as a general monitoring surface, second workboard, or backlog sweep.

One counterintuitive result is that correct silence became a control behavior. If the governing work record is truthful and no recipient must act, sending a completion note can create false motion. Message evidence shows repeated manager closeouts where `Reply needed: NO` was honored and no direct message was sent.

The system also avoided awareness-only copies because extra messages are not harmless notifications. For an AI employee, every inbound message can become a work trigger unless the institution gives it explicit no-action or no-reply semantics.

5.7 Playbooks As Operating Doctrine

Playbooks are reusable doctrine, not optional retrieved context. They own workflow logic, domain-specific acceptance criteria, required inputs and outputs, source truth, replay standards, and recurring review steps.

Playbooks were not simply pre-authored instructions. They became the durable landing zone for reusable lessons discovered during work. Case-specific facts remained in work records; reusable procedure moved into playbooks; authority remained in role text. This placement discipline is what turned ad hoc correction into institutional learning.

This differs from generic documentation in three ways:

1. Runtime doctrine must be listed and materially relevant playbooks must be read before substantive action.
2. Reusable learning must be promoted to the correct durable layer rather than left in a ticket or message thread.
3. Playbook changes are institutional mutations, not ad hoc prompt edits.

Because role text sits above playbooks in the authority stack, reusable procedures should not be copied wholesale into roles. Stale workflow embedded in role text can override newer playbook doctrine. Mature role design keeps roles focused on authority, interface, outputs, and escalation, while playbooks own detailed operating procedure.

The doctrine became more procedural exactly where failures recurred: source freshness, packet completeness, activation, no-reply handoffs, replay linkage, and doctrine editing.

The organization also learned that long playbooks and role texts are necessary for explicit model control but risky to rewrite wholesale. This motivated targeted old-string/new-string doctrine edits, which preserve local explicitness and reduce accidental compression or restyling. This report treats targeted doctrine patching as mechanism evidence; quantitative edit-error, context-cost, and recurrence values are not reported as bounded values.

5.8 Evidence Surfaces And Decision Packets

High-skill work cannot be reviewed if it is only plausible prose. Weather and market research therefore had to become verifier-reconstructible packets: source basis, settlement mapping, uncertainty, scenario reasoning, probability or decision structure, confidence, contradiction checks, freshness, and no-action triggers.

The generalizable mechanism is not weather-specific. In technical AI organizations, downstream review benefits from artifacts a second employee can verify without redoing the producer's entire job. The producer's artifact should expose enough evidence and assumptions for a verifier to accept, reject, repair, reroute, or decline.

5.9 Verifier Topology

The organization used verifier topology rather than verifier prompting. The basic topology was:

1. Creator produces an artifact.
2. Functional verifier accepts, declines, repairs, or reroutes.
3. Executor acts only inside accepted authority.
4. Manager verifies closure and durable learning.

Figure 4. Creator -> Verifier -> Executor -> Manager -> Replay.

```
creator artifact
  -> functional verifier decision
  -> authorized execution or no-action
  -> manager closure decision
  -> replay against outcome truth
```

Takeaway: review becomes an authority-bearing route through state decisions, not a prose critique step.

This made verification an authority-bearing state transition. A verifier message did not ask for "thoughts." It asked for a governing decision. Message evidence showed verifier responses ending in approval, decline, repair handoff, reroute, or manager review. Some approvals changed the downstream executable envelope rather than rubber-stamping the upstream proposal. Some declines explicitly withheld execution authorization even when the packet quality passed.

Verifier topology separates artifact review from execution authorization. A packet can be reconstructible yet still fail the action gate. In those cases, the verifier can decline execution, request repair, reroute the work, request manager review, or narrow the executable envelope.

A sanitized verifier decision can be represented without exposing market strategy or raw operational records:

```
Work item: [redacted market-path episode]
Decision: Repair required
Reason: settlement mapping incomplete; source freshness not established
Next owner: research employee
Next action: refresh source basis and resubmit packet
Execution authorized: no
```

5.10 No-Action States

No-action became first-class. Valid outcomes included no-send, decline, repair, stand-down, blocked, already-done, explicit zero, no replay needed, and case-only learning. These outcomes are not failures by default. They

are often the organization's correct output.

Explicit-zero is a verified no-action state: the organization records that no live child path, downstream owner, or executable action remains. It is completed work only because the governing record is truthful and accepted, not because an outcome was proven correct.

Figure 5. Work Item Lifecycle.

```
trigger received
  -> inbox state: actionable / superseded / already done
  -> governing record identified
  -> planning gate: authority + doctrine + evidence
  -> primary outcome:
       completed / handoff required / blocked / needs review / needs clarification
  -> closure / no-action / replay / mutation review
```

Takeaway: the lifecycle treats stopping, handoff, review, replay, and mutation review as explicit institutional states.

This matters because AI systems tend to reward visible action. In high-skill work, value often comes from refusing unsupported action. The organization therefore had to make stopping auditable: why no action was correct, who had authority to decide, whether review was required, whether replay was needed, and who owned the next step if any.

Each substantive handling cycle ended in exactly one primary outcome:

Primary outcome	Meaning
Completed	Work is done, including valid no-change or no-action completion
Handoff required	Another owner must act
Blocked	A real dependency prevents progress
Needs review	A verifier or manager must decide
Needs clarification	A narrow missing fact or ambiguity blocks correct action

5.11 Replay And Institutional Mutation

Replay connects outcome truth to organizational learning. A replay record reconstructs the original path, active role/playbook context, decision artifact, action or no-action, later outcome truth, process correctness, outcome correctness, and learning destination.

The important replay detail is historical reconstruction. Replay asks what role contract and playbook versions governed the employee at the time, not what the latest doctrine says now. This prevents the organization from judging old work against rules that were not in force at the time and makes expected-versus-realized review possible.

Learning destinations include:

- role text;
- playbook;
- tool access;
- staffing;
- topology;
- workboard rule;
- message rule;
- case-only record.

This is the adaptive institution loop:

> evidence -> classification -> authorized mutation -> activation -> recurrence check.

Reflection is insufficient because it can stay private to one model invocation. Institutional learning is complete only when the reusable fix lands in the correct durable layer or is intentionally classified as case-only.

A sanitized replay record can be represented as:

```
Original work path: [redacted market-path episode]
Original decision artifact: evidence packet plus verifier decision
Role version at decision time: [redacted role contract version]
Playbook version at decision time: [redacted review/playbook version]
Action state: action / no-action / stand-down
Outcome truth: later observed result or settlement state
Expected-vs-realized review: process-correct / process-gap / outcome-gap / unclear
Learning destination: playbook / role / tool / workboard / message / topology / case-only
Mutation status: proposed / applied / rejected / deferred / case-only
```

5.12 Person-Role-Aware Tool Surface

The tool surface was split into two planes:

1. **Work-tool plane:** domain tools used to gather evidence, inspect markets, compute, or execute.
2. **Institutional-control plane:** tools for employees, messages, profiles, playbooks, role edits, tool edits, hiring, termination, and doctrine mutation.

The important idea is not the particular implementation. It is that organization-grade tool use must be interpreted through employee identity, role authority, governing work state, and doctrine. MCP-style tools expose capability. An organization layer decides when that capability may be used and what durable state change it creates.

In this sense, the tool surface functioned as an executable authority model. An authenticated employee principal entered the organization through a caller-dependent surface: ordinary employees could read profiles, inspect inboxes, read playbooks, message coworkers, and perform assigned work; managers or owner-level callers could perform durable administration such as role edits, tool-set changes, hiring, termination, and playbook mutation. Profile, role, tool-set, inbox, work-record, and playbook reads were intentionally separated because each surface answers a different authority question.

The table abstracts the public-facing surface into institutional semantics; it is not a complete tool inventory.

Surface	Regular employee	Manager / owner	Institutional meaning
Profile / role reads	yes	yes	identity and authority are inspectable
Inbox / message tools	yes	yes	communication is work activation
Playbook reads	yes	yes	doctrine is readable at runtime
Role/tool/playbook mutation	no	yes	durable institutional change is privileged
Hiring / termination	no	yes	employees are institutional principals

Tool outputs were also shaped for institutional use: list and inbox surfaces were paginated, outputs were minimized, message status was exposed, and role/playbook version reads supported audit and replay.

For communication, the institutional tool surface exposed sender identity, message status, timestamps, and response/failure fields so inbox reads could support obligation freshness rather than become a general chat archive.

A sanitized authority example is:

```
Employee state: can inspect evidence and draft a recommendation
Tool capability: can see an action-capable work surface
Role authority: cannot authorize downstream execution
Required gate: verifier acceptance and authorized envelope
Valid outcome: record needs-review or handoff state; message the authorized next owner
Invalid outcome: execute because the tool surface made action technically possible
```

This is the concrete version of "tool access is not authority." Capability is a necessary substrate, but the valid state mutation depends on role, work record, verifier gate, and manager or owner boundary.

6. Evaluation Framework

The evaluation asks:

> Did the organization produce accountable, authorized, evidence-backed, outcome-reconciled work, and could reliability-relevant failures be audited, bounded, or repaired?

It deliberately avoids using profit, number of employees, number of tools, prompt length, or message volume as primary evidence. Those are context. They do not prove institutional control.

6.1 Metric Families

The metrics are not task-success metrics. They are institution-state metrics: they ask whether the organization made the right work truth visible and actionable.

Metric status is reported in four tiers: `defined` means the metric is conceptually specified; `extractable` means source artifacts are known but no denominator-labeled value is reported; `mechanism evidence extracted` means sanitized cases support the mechanism; `bounded value extracted` means this report includes a numerator, denominator, sample label, and window. No denominator means no bounded value.

Metric	What it checks	Evidence/status	Current reported value	Main limitation
Ownership validity	owner and next action/blocker	P1/B1 spot-checks; S1 closeouts; bounded value extracted	P1 late-Mar/early-Apr: 21/30 owner, 18/30 next; S1 Apr 26: 20/20 both; B1 non-S1/R1/V1: 62/75 owner, 51/75 next	P1/B1 shallow; S1 selected
Authority compliance	role, doctrine, owner/manager, and state permission	mechanism evidence extracted	no rate reported	mechanism evidence only
Evidence sufficiency	packet reviewability	mechanism evidence extracted	repairs and declines observed; no packet census	no denominator
Verifier gate quality	stateful accept, repair, reroute, decline, or review	mechanism evidence extracted	S1 lower bounds: at least 16 approvals and at least 10 repairs	lower bounds, not rates
Closure validity	authority-accepted final state	S1 closeouts; bounded value extracted	S1 Apr 26: 20/20 manager-reviewed; 0/20 complete-but-open	selected window

Metric	What it checks	Evidence/status	Current reported value	Main limitation
Workboard frontier validity	parent, child, activation, closure, and replay state agreement	mechanism evidence extracted	no rate reported	duplicate-parent case is not denominator-coded
No-action quality	evidence basis and no-replay treatment	S1 no-execution paths; bounded value extracted	S1 Apr 26: 6/6	selected window; not correctness
Message trigger auditability	exact ask, reply flag, and stale suppression	M1 packets, S1 closeouts, and S1 live umbrella; mechanism evidence extracted	S1 Apr 26: 10 direct activation messages recorded for 20 child records; S1 Apr 26: 20/20 final closeouts sent no downstream message when no action remained; M1 packet examples show stale/no-change suppression	S1 activation count comes from workboard/umbrella evidence, not a full raw inbox census
Replay completion	replay or no-replay rationale	S1 replay/final records; bounded value extracted	S1 Apr 26: 14/14 replay closed; 6/6 no-replay	no recurrence rate
Topology routability	lane, chain, tool surface, and activation path identify owner	mechanism evidence extracted	no rate reported	mechanism evidence only
Tool parity	required tool surface before activation	mechanism evidence extracted	no denominator or time-to-parity value	mechanism-level evidence only; no bounded estimate reported
Durable mutation latency	time from learning to durable-layer change	defined	no value reported	defined but not estimated in this study
Recurrence after mutation	same failure after activated change	defined	no value reported	defined but not estimated in this study
Human intervention burden	owner or human repair frequency	mechanism evidence extracted	S1 includes one opening owner directive; no rate	not global burden

Appendix H records detailed evidence status and coding guidance for metrics this study defines but does not estimate. The main Results section keeps only denominator-labeled counts and short evidence-basis notes.

7. Results

This section reports defensible findings from extracted evidence. It does not invent aggregate organization-wide rates. Quantitative rows use bounded samples: the primary window is `S1`, a 20-child live sprint paired with its retro; `R1` and `V1` are used only as labeled supporting windows.

The Results section measures institutional auditability and controllability, not trading performance, forecasting accuracy, or global decision quality. Counts are reported only for bounded windows with denominator-labeled extraction. Where evidence is qualitative or mechanism-level, the text does not infer aggregate rates.

Across extracted mechanism packets, institutional state changed downstream behavior in ways that ordinary record-completeness tables do not fully show:

Pillar	Mechanism	Observed behavior change	Evidence type	Claim type	Limitation
State	Manager closure	Apparently complete work was kept open, rerouted, or closed only after final disposition.	S1 closeout audit	bounded metric	selected mature window
Authority	Verifier gate	Approval, repair, decline, reroute, and changed execution envelope became state decisions.	S1 verifier comments	mechanism evidence	lower bounds, not rates
Communication	Message staleness	No-change and superseded triggers were suppressed before action.	M1 packet examples	extracted mechanism	not inbox census
State	Workboard compilation	Broad intent became child paths, gates, activations, and replay obligations.	S1/R1/V1 sprint structure	mechanism evidence	no corpus-wide rate
Learning	Replay classification	Live closure, no-replay rationale, and doctrine follow-up were separated.	S1 replay and retro records	bounded metric	no recurrence rate
Authority	Tool-parity-gated activation	Activation waited until role, tool, routing, and activation surfaces aligned.	Case I mechanism evidence	extracted mechanism	no recurrence or safety claim

7.1 Finding 1: State Control Made Work Truth Auditable

Evidence basis: bounded metric audit.

State control converted ambiguous conversational residue into auditable institutional state. Work that looked substantively complete still required an accepted final state, named dependency, reroute, or explicit no-action treatment on the governing record.

A bounded runtime-contract compliance audit on the S1 final manager-closeout sample found that 20/20 final closeout records included fixed planning and execution stamps, inbox check state, materially relevant playbooks

read, exactly one primary outcome, next owner/action fields, and a messages-sent field. In the same sample, 20/20 final manager closeouts sent no direct message when no downstream action remained, and 6/6 final no-execution paths were audited as valid no-action with explicit no-replay treatment. These are not global rates; they show that the mature contract made closure, no-action, and silence visible on the governing record.

These presence metrics measure record completeness and auditability, not decision correctness. A field was counted when it was present in the fixed record structure; the count does not by itself prove the underlying decision was correct.

Runtime-contract compliance field	S1 final closeout denominator	Count
Planning and execution stamps present	20 final closeout records	20/20
Inbox state, playbooks read, primary outcome, next owner/action, and messages-sent fields present	20 final closeout records	20/20
Final closeouts with no direct message sent when no action remained	20 final closeout records	20/20
No-execution paths audited as valid no-action	6 final no-execution paths	6/6

The same evidence supports an institutional distinction between completion and closure:

- **completion**: a worker's assigned output is finished;
- **closure**: an authorized verifier has accepted the state and recorded final truth.

Work records showed that completion and closure diverged. Some work appeared substantively done but remained open, review-ready, or structurally incomplete. The manager-verification intervention required manager-touched records to end as one of: closed, kept open for a named dependency, rejected with the missing piece, or rerouted to the next owner.

Metric	Status	Value
Complete-but-open exceptions after final manager closeout	S1 live child tickets	0/20 observed
Manager touches with final disposition after intervention	S1 live child tickets	20/20 final live dispositions manager-reviewed; 2 pre-final manager reroutes bounded premature finality
Review-ready records left without state transition	S1 live child tickets	0/20 observed after final manager verification

Limitation. These are record-completeness and auditability metrics from a selected mature window, not correctness or full deployment rates.

7.2 Finding 2: Authority Control Turned Review Into State Decisions

Evidence basis: mechanism evidence with bounded lower-bound counts.

Verifier evidence shows that review messages did not ask for generic critique. They asked the verifier to make a governing decision. The verifier could approve, decline, return repair items, reroute, or request manager review. Approval could also modify the executable envelope rather than rubber-stamping the upstream proposal. This made authority a topology property rather than a tone of reviewer feedback.

Verifier topology separated two questions that agent systems often collapse: "Is the artifact reconstructible?" and "Is downstream action authorized?" A packet could be reviewable while still failing the action gate, causing a verifier to decline execution, request repair, reroute, request manager review, or narrow the executable envelope.

Rows using "at least" are conservative lower bounds from extracted comments, not event-rate estimates over the full deployment.

Verifier outcome	Status	Observed count or disposition
Approve	S1 functional-verifier comments	At least 16 approval events observed
Repair required	S1 functional-verifier comments	At least 10 repair-required decisions across at least 9 child paths
Decline/no execution	S1 final live child dispositions	6/20 final no-execution paths
Reroute	S1 comments	4 observed reroutes: 2 execution-stage and 2 manager-stage
Needs manager review	S1 live child tickets	20/20 final dispositions routed through manager verification
Approval changed upstream proposed action	S1 comments	3 clear changed-envelope approvals

Authority control also constrained execution. A proposed action was not executable merely because a worker generated it or a tool could perform it. The action had to pass through role authority, verifier gates, and manager closure. This is the paper's main distinction from agent-level guardrail framing: the gate is attached to an organizational state transition, not only to a model output or tool call.

Limitation. The verifier counts are conservative observed lower bounds in extracted comments, not full verifier-event rates.

7.3 Finding 3: False-Motion Control Made Stale, Duplicate, And Non-Material Work Suppressible

Evidence basis: mechanism evidence plus bounded closeout counts.

Communication control made false motion suppressible. Duplicate state sync, no-change updates, waiting loops, fake-active live work, stale parent umbrellas, and broad sprint triggers can all look like activity while failing to change organizational truth; the message protocol and inbox checks made no-reply, stale suppression, and narrowed activation auditable outcomes.

Metric	Status	Value
Manager-review messages with no-reply expectation	S1 closeout records plus message packet sample	Observed as a closeout pattern; no global inbox rate claimed
Closeouts with no direct message sent	S1 live child manager closeouts	20/20
Direct activation messages	S1 live umbrella	10 direct activation messages recorded for 20 child records
Self-trigger no-change suppressions	260-row inbox packet sample	1 sanitized packet
Superseded/stale unauthorized-message suppressions	260-row inbox packet sample	2 sanitized packets
Broad sprint triggers narrowed into child activations	S1/R1/V1 sprint structure	Mechanism observed; no corpus-wide rate claimed

Limitation. These examples show false-motion suppression in bounded samples; they do not establish loop elimination.

The S1 live umbrella recorded 10 direct activation messages for 20 child records; this supports the claim that handoff was treated as an activation event, not merely assignment metadata. The activation count comes from workboard/umbrella evidence, not a full raw inbox census.

7.4 Finding 4: Learning Control Separated Stopping, Replay, And Doctrine Change

Evidence basis: bounded metric audit.

Learning control separated stopping, live closure, delayed outcome truth, and durable doctrine change. In the S1 no-execution paths, no-action was not treated as missing work: it was made reviewable through evidence basis, authority basis, and explicit replay or no-replay treatment. In the strongest form, explicit zero is verified non-action: the organization records that no live path remains and manager closure accepts that state.

No-action metric	Status	Value
No-action records by type	S1 live child tickets	6/20 final no-execution paths: 4 verifier declines and 2 no-send/weak-edge decisions
No-action records with evidence basis	S1 no-execution paths	6/6
No-action outcomes later revisited or classified	S1 comments and retro	2 no-action/decline states were rerouted before final closure; final no-execution paths were classified no-replay-required

Replay separated live-work closure from delayed outcome truth. An executed or no-action path could be closed operationally while a replay child or retro surface remained open for later settlement, fill, cancellation, or outcome review.

The important learning mechanism was destination classification. A replay could conclude that learning belonged in a playbook, role, tool surface, staffing/topology, workboard rule, message protocol, or only the case record. This controlled two common failure modes: over-promoting one-off facts into doctrine and under-promoting repeated failures into dead comments.

Replay metric	Status	Value
Ordered paths with replay complete	S1 replay tickets	14/14 closed under the retro umbrella
No-execution paths with explicit no-replay rationale	S1 no-execution paths	6/6
Replay findings promoted to durable mutation	S1 retro closeout	0 immediate durable patches; 1 research/doctrine follow-up artifact
Replay findings classified case-only	S1 retro closeout	Ticket-level replay records were closed, but per-ticket destination split was not separately coded

Limitation. The no-action and replay counts show reviewability and closure treatment, not outcome correctness or recurrence reduction.

7.5 Finding 5: Topology Control Made The Organization Routable

Evidence basis: mechanism evidence.

The organization did not gain auditable control merely by adding more agents. It gained routable institutional control when agent seats were placed into topology: role families, pods or lanes, default downstream chains,

verifier and executor separation, manager finality, capacity/overflow concepts, production/research separation, and synchronized truth surfaces.

This finding is reported as mechanism evidence, not a topology-performance result. The observed pattern is that structural change was only operational when role text, profile or roster truth, tool access, work records, and activation messages agreed. A role could exist in design but remain ineffective if it lacked the right tool surface or activation path. A tool-capable employee could still lack authority if the role, verifier gate, or governing record did not authorize action.

Topology mechanism	Control function	Failure exposed or controlled	Evidence status
Single-manager finality	One authority boundary for closure and durable mutation	split-brain doctrine, people, or closure changes	mechanism evidence
Role families	Repeated seats share interface contracts	bespoke prompts with incompatible handoff expectations	mechanism evidence
Pods or lanes	Upstream, review, execution, and replay paths are routable before work starts	every worker infers the next collaborator from context	mechanism evidence
Default downstream chains	Normal handoffs are structural, exceptions are recorded	hidden local reroutes and orphaned ownership	mechanism evidence
Capacity, overflow, and failover	Overload or unavailable owners become explicit routing conditions	silent accumulation of stalled AI work	mechanism evidence; bounded counts not reported
Production/research separation	Method learning cannot bypass live authority	research output becomes unauthorized production action	mechanism evidence
Truth-surface synchronization	Role text, profile, tool set, work record, playbook, and activation agree before a change is live	a worker exists on paper but cannot operate correctly	mechanism evidence

Limitation. These are topology mechanisms observed in one organization; the report does not measure topology performance or claim a generally optimal structure.

8. Mechanism Case Studies

The case studies use sanitized labels. They are organized to show the main institutional-control mechanisms across state, authority, communication, and learning, not private operational details.

Case	Failure	Org diagnosis	Interv.	Changed artifact	Claim type	Evidence basis	Limitation
A	complete but open	closure invalid	manager queue	workboard, manager role	bounded metric	S1 closeout audit	no full before/after rate
B	stale or duplicate triggers	message lifecycle failure	inbox pre-check	message protocol	extracted mechanism	M1 packets, S1 closeouts	not inbox census

Case	Failure	Org diagnosis	Interv.	Changed artifact	Claim type	Evidence basis	Limitation
C	plausible prose	artifact and action gates collapsed	packet and action gates	playbook, verifier role	extracted mechanism	verifier comments	domain fields abstracted
D	doctrine drift risk	durable-law mutation risk	anchored patch	role and playbook tools	mechanism evidence	sanitized patch pattern	no cost, error, or recurrence claim
E	lesson trapped in retro	wrong landing layer	replay classification	replay and doctrine process	extracted mechanism	replay and retro records	no recurrence rate
F	unclear routing	not routable	role lanes	topology	extracted mechanism	topology cases	no optimum claim
G	replay without patch	learning gap visible	negative replay classification	replay record	negative mechanism	S1 retro closeout	no immediate mutation
H	broad sprint ask	frontier not compiled	child-path structure	workboard	extracted mechanism	sprint structure	selected windows only
I	activation with parity gap	surfaces not aligned	activation gate	role, tool, routing, activation surfaces	extracted mechanism	tool-parity activation case	no recurrence or safety claim
J	repeated local fixes	correction not institutionalized	authorized artifact promotion	role, work, message, verifier, replay, tool, and playbook forms	mechanism synthesis	cases	no recurrence claim

8.1 Case A: Manager Verification Queue And Atomic Closeout

Initial failure. Work appeared substantively complete, but the governing record did not reach a valid final state. Some items were complete-but-open; others were review-ready but still mixed with active work. The organization could tell a story of completion, but the workboard did not carry closure truth.

Organization diagnosis. A worker-level view would ask whether the assigned output was produced. The organizational failure was invalid closure: no authorized owner had accepted the final state, checked child/replay obligations, and left no ambiguous next action.

Intervention. The organization introduced manager verification queue discipline. A manager review cycle had to end in exactly one state: closed, kept open for named dependency, rejected with missing piece, or rerouted to the exact next owner.

Changed durable layer. Workboard process, manager role, closure doctrine, and runtime-contract rules.

Evidence artifact. Extracted work-record cases show complete-but-open and umbrella-closeout failures, plus follow-through where remaining cases were closed or left open for named dependencies.

General principle. Completion is not closure. AI organizations benefit from closure validity as a first-class institutional state.

Limitation. Available evidence supports the mechanism but does not include a full before/after closure-validity

rate.

8.2 Case B: Message Trigger Discipline And Stale/No-Reply Suppression

Initial failure. Assignments, comments, and tracker fields did not reliably wake the correct employee. Later, direct messages themselves created a new problem: old or duplicate messages could cause repeated review, acknowledgment loops, or no-change state churn.

Organization diagnosis. Multi-agent chat treats communication as collaboration. The organizational failure was message lifecycle ambiguity: communication needed to be a work-transfer protocol with lifecycle state.

Intervention. Work messages were required to include a work item, exact ask, exact next action, and reply-needed flag. Inbox checks classified messages as actionable, superseded, or already done. Routine replies were suppressed when no recipient needed to act.

Changed durable layer. Message protocol, transparent inbox tools, runtime-contract planning, loop-prevention doctrine.

Evidence artifact. Message evidence shows manager review requests with no-reply expectation, closeouts without direct replies, self-triggers that stopped after no-change checks, and execution requests rerouted because current gate truth no longer authorized action.

General principle. A message is not chat. It is an auditable trigger for one focused work obligation. Correct silence can be part of accountable, controllable AI labor.

Limitation. Message metrics combine two evidence types: work-record routing counts from S1 and packet examples from a bounded inbox sample. They support the mechanism, but they are not a global organization-wide loop-rate estimate.

8.3 Case C: From Plausible Prose To Verifier-Reconstructible Artifact And Action Gate

Initial failure. A domain specialist could produce plausible narrative analysis, but downstream review needed reconstructible fields: source basis, freshness, settlement mapping, uncertainty, probability or decision structure, confidence, contradiction checks, and no-action triggers.

Organization diagnosis. Fluency hid two missing institutional gates: evidence sufficiency for review and action authorization for downstream execution.

Intervention. Research outputs became structured packets. Review playbooks defined packet quality gates and action gates. Verifier messages asked for stateful decisions: approve, repair, reroute, decline, or request manager review.

Changed durable layer. Playbook doctrine, verifier role, work-record acceptance criteria.

Evidence artifact. Extracted work records and verifier inbox samples show packet-gate failures, repair routing, approvals that modified the downstream envelope, and declines that explicitly withheld execution authorization. The case shows two gates: whether the artifact was reconstructible and whether downstream action was authorized.

General principle. Technical AI work benefits from verifier-reconstructible artifacts, not persuasive prose, and from action gates that treat a good artifact as insufficient for execution authority.

Limitation. Domain-specific packet fields are abstracted for readers outside trading.

8.4 Case D: Targeted Doctrine Patch Instead Of Whole Rewrite

Initial failure. As role contracts and playbooks became more explicit, they also became longer. Whole-document rewrites for small changes risked compression, omitted clauses, restyling, accidental scope changes, and higher context and review cost.

Organization diagnosis. Prompt iteration treated doctrine as a single mutable blob. The organizational problem was durable-law mutation: small changes needed to land without damaging unrelated clauses.

Intervention. The system introduced targeted replacement edits for role and playbook text. A small correction could specify the old string and the new string, with optional replace-all only when appropriate. This made doctrine mutation local, reviewable, and less likely to silently rewrite unrelated policy.

Changed durable layer. Role/playbook mutation tools and doctrine maintenance process.

Evidence artifact. Sanitized mechanism evidence identifies anchored replacement behavior. A local-patch pattern is:

```
Old clause: if another employee should act, update the work record.
New clause: if another employee must act, send a direct message naming the work item, exact ask, exact ne
Patch rule: apply only when the old clause is found exactly once.
```

An ambiguous-anchor pattern is:

```
Requested old clause: "send a direct message"
Result: rejected because the clause appeared in multiple doctrine sections.
Required repair: narrow the anchor or explicitly authorize replace-all.
```

General principle. AI organizations benefit from doctrine surgery, not only prompt rewriting.

Limitation. This case supports the mechanism but not quantitative claims about context-cost savings, lower edit error, or downstream recurrence reduction.

8.5 Case E: Replay-To-Institutional-Mutation

Initial failure. Completed work could produce outcome truth later, but the lesson could remain trapped in a ticket, message, or one worker's context. Conversely, one-off outcome facts could be over-promoted into general doctrine.

Organization diagnosis. Reflection and memory can generate lessons, but they do not decide where the organization should change.

Intervention. Replay records reconstructed the original path and classified learning destination: role, playbook, tool, staffing, topology, workboard, message, or case-only. The stronger replay form also records the role and playbook versions active at decision time, so expected-versus-realized review does not judge old work only by later doctrine. Durable changes required manager or owner authority.

Changed durable layer. Replay doctrine, role/playbook/tool/topology mutation process, manager authority boundary.

Evidence artifact. Extracted replay and retro cases show scoped learning, case-only decisions, and selected durable-change mechanisms, while S1 shows a negative case where replay did not immediately become durable mutation.

General principle. Learning is incomplete until the durable fix lands in the correct organizational layer.

Limitation. Recurrence after mutation is only partially extracted. This report does not claim that any replay-driven change removed a failure class.

8.6 Case F: Adaptive Organization Through Role, Tool, Staffing, And Topology Change

Initial failure. Adding more AI workers increased routing ambiguity and tool/role mismatch. A worker could exist without the right institutional surface, or a pod could route work without the right verifier/executor chain.

Organization diagnosis. Adding specialists as prompts did not define staffing, authority, tool parity, activation, or routing topology.

Intervention. The organization developed role families, pods, default chains, manager-only durable changes, tool parity checks, and activation discipline.

Changed durable layer. Organization topology, employee/tool lifecycle, role-family contracts, workboard routing.

Evidence artifact. Extracted organization rollout and tool-parity cases show that staffing or tool repair could be the correct institutional fix, not another prompt revision.

General principle. An AI-staffed organization prototype can adapt institutionally: it can change people, doctrine, tools, routing, and topology.

Limitation. This case is presented as mechanism discovery from one organization, not evidence of a generally valid topology.

8.7 Case G: Replay Without Immediate Durable Mutation

Initial failure. A replay/retro closeout produced a research/doctrine follow-up artifact but did not immediately create a durable playbook, role, tool, staffing, topology, workboard, or message-protocol mutation.

Organization diagnosis. Institutional control made the learning gap visible, but visibility did not automatically mean the organization had completed durable learning. This is a negative case against overclaiming replay as recurrence reduction.

Intervention. Replay classification separated the case-level finding from immediate durable mutation.

Changed durable layer. Replay record and retro closeout state.

Evidence artifact. The S1 retro closeout recorded 0 immediate durable patches and 1 research/doctrine follow-up artifact.

General principle. Replay can separate outcome truth from live closure, but durable organizational learning remains incomplete until the mutation destination is resolved and the change is activated.

Limitation. This report does not measure whether the follow-up artifact later became durable doctrine or reduced recurrence.

8.8 Case H: Sprint Compilation Into Child Path Work

Initial failure. A broad owner request could sound operationally clear while leaving the actual AI labor frontier underspecified: which paths existed, who owned each path, which gate applied, which employee was activated, which paths required replay, and when the parent could close.

Organization diagnosis. A workflow view could treat the sprint as a set of tasks. The organizational failure was that owner intent had not been compiled into durable state that every employee could route, verify, close, or replay without relying on private context.

Intervention. The manager compiled broad intent into a live umbrella, retro umbrella, child path records, shared control fields, direct activation messages, verifier gates, no-action or closeout states, and replay obligations.

```
owner intent
  -> live umbrella
  -> retro umbrella
  -> child path records
  -> direct activations
  -> verifier gates
  -> close / no-action / replay
```

Changed durable layer. Workboard topology, message activation, manager closeout, and replay surface.

Evidence artifact. S1/R1/V1 sprint windows show live and retro pairings, child path records, manager-reviewed final dispositions, and ordered-path replay separation. The broader P1/B1 spot-check is used only as context, not as a full workboard-frontier rate.

General principle. The workboard is not a passive tracker. It is the institutional state machine that makes vague intent executable by bounded AI employees.

Limitation. This report shows this mechanism in selected and spot-check samples, but it does not prove the compilation topology is optimal or sufficient for all domains.

8.9 Case I: Tool-Parity-Gated Activation

Case F describes the general topology pattern: role families, lanes, default chains, tool parity, and activation discipline. Case I shows one extracted activation-gating lifecycle.

Initial failure. A topology change could create live employees with partial capability, stale routing truth, or ambiguous activation. A new role could have contract text while still lacking the tool surface or activation path needed for its assigned work.

Organization diagnosis. The activation boundary failed unless role, tool, routing, and activation surfaces all agreed.

Intervention. Activation was held until required role, tool, routing, and activation surfaces were directly evidenced as aligned.

Changed durable layer. Organization topology, role text, playbook/routing doctrine, tool/profile truth, activation protocol, and workboard cutover state.

Evidence artifact. Extracted mechanism evidence shows tool-class parity and activation-gating behavior. Sensitive tool details are omitted from public artifacts.

General principle. A role or tool surface is not live institutional capacity until the activation path agrees with the governing routing state.

Limitation. This case does not show recurrence effects, reliability effects, or a general rollout method. It shows one source-backed mechanism for holding activation until required role, tool, routing, and activation surfaces aligned.

8.10 Case J: Durable Artifact Growth From Repeated Failure

Initial failure. Repeated work initially relied on bespoke prompts, comments, or case-specific work text. A manager or employee could correct a local problem, but the same missing structure could reappear because the correction had not become a durable institutional artifact.

Organization diagnosis. The failure was not only weak instruction following. It was that one-off correction was not durable doctrine, authority, state, or activation protocol. The organization needed a way to decide which repeated lessons belonged in role contracts, work-record fields, message rules, verifier decision forms, playbooks, replay doctrine, or tool surfaces.

Intervention. Under manager or owner authority, recurring fixes were promoted into durable artifact forms. Repeated role ambiguity became role-family contracts. Repeated case-truth ambiguity became work-record fields. Missed handoffs and false motion became message rules. Plausible but unauthorized artifacts became verifier decision forms. Outcome-learning gaps became replay doctrine. Capability gaps became tool-surface changes.

Changed durable layer. Role contracts, workboard/work-record structure, message protocol, verifier gate fields, playbook doctrine, replay process, and tool surface.

Evidence artifact. The intervention timeline, replay cases, targeted doctrine patching, workboard compilation, message discipline, verifier gates, and tool-parity case all show local failures being routed toward durable artifact changes rather than left as isolated conversation.

General principle. Durable artifact forms are institutional learning outputs. The contribution is not that complete schemas existed at the start, but that runtime-bounded work made missing structure visible enough for authorized promotion.

Limitation. This is mechanism evidence. It does not show autonomous self-improvement or a measured recurrence reduction.

9. Discussion

9.1 Main Interpretation

The main interpretation is that this field study suggests high-skill AI labor may need a control layer around agents. The organization did not establish global decision-quality improvement merely because workers received more instructions. It made reliability-relevant failures more visible and controllable when failures were converted into durable primitives: role authority, work-state truth, message lifecycle, evidence gates, no-action states, replay, and authorized mutation.

State, authority, communication, and learning are not claimed to be a complete taxonomy of AI organizations. They are the four control surfaces that became visible in this field study and that organized the observed failure-to-intervention pattern.

This is why "AI organization" should not be used as a loose metaphor for a group of agents. It should name a concrete institutional architecture.

The deployment was AI-staffed in daily operational labor, but not human-free. The human owner remained the boundary for goals, acceptable risk, high-level design direction, and publication judgment. In the S1 window, the bounded sample includes one sprint-opening owner directive, 20 manager-reviewed final closeouts, at least 9/20 child paths requiring packet/source repair before final decision, and no direct staffing/tool/topology mutation. This is evidence about a human-owned, AI-staffed institution, not a claim of fully unsupervised replacement.

9.2 Falsifiers And Boundary Conditions

The thesis would weaken under evidence that:

- improvements came mostly from model upgrades rather than organizational interventions;
- human intervention remained the hidden driver of correctness;
- work records became compliant but decisions did not improve;
- no-action states hid indecision rather than valid restraint;
- replay did not reduce recurrence or improve doctrine;
- the mechanisms failed outside prediction-market trading;
- manager finality became an unscalable bottleneck.

The paper should welcome these tests. Institutional control is an empirical claim, not a slogan.

9.3 What Current Agent Approaches Miss

Popular focus	Why useful	Why incomplete
Better models	improves reasoning and generation	does not create ownership or closure
Tool use	expands action space	does not define authority
Memory	preserves context	does not create shared doctrine
Handoffs	moves control between agents	does not guarantee accountable work transfer
Workflow graphs	control sequence and persistence	do not define institutional acceptance
Guardrails	validate inputs, outputs, or tool calls	do not replace role authority and verifier topology
Multi-agent teams	distribute expertise	can still rely on conversation as state
Human-in-loop	adds oversight	can hide missing organizational design

The paper's claim is stronger than "use multiple agents with tools." It is "make the organization the unit of design and evaluation."

9.4 Practical Design Rules For AI Organizations

The practical design rules are:

- The model is not the system of record.
- Tool access is not authority.
- Messages are work triggers, not chat.
- Review must produce a state decision, not commentary.
- No-action must be a valid audited output.
- Learning is incomplete until it changes the right durable layer or is classified case-only.
- Humans move upward into institutional design: goals, authority, evidence standards, verification topology, replay rules, and mutation rights.

9.5 Humanlike Interface, Machine-Native Control

The paper uses terms such as employee, manager, sprint, review, handoff, and retro because they are legible handles for humans and for other AI workers. Those terms should not be read as anthropomorphic evidence or as roleplay. They are the interface layer.

The control layer is machine-native: authority order, role boundaries, tool surfaces, work-record state, child-frontier topology, message freshness, verifier gates, no-action states, fixed stamps, replay linkage, and manager-authorized mutation. A title does not grant authority unless role text, tool access, governing record state, and activation all support it. A manager review is not a courtesy comment; it is a state transition. A sprint kickoff is not a social ritual; it is compiled into live and retro umbrellas, child paths, gates, and exact activations.

This interface/control separation is a useful way to read the whole system. The organization can look familiar, but its auditability comes from explicit state machinery.

9.6 Prototype System As Reference Implementation

The prototype system used in this study is best treated as one reference implementation of broader organization-protocol semantics. The paper does not imply that others must adopt the same product, implementation substrate, folder structure, or internal implementation.

The portable primitives are:

- employee principal;
- role contract;
- governing work record;
- work message;
- playbook doctrine;
- verifier gate;
- no-action state;
- replay record;
- doctrine patch;
- institutional mutation.

The standardization opportunity is analogous to but above MCP. MCP gives AI applications a standard way to access tools, resources, and prompts. An organization protocol would give AI employees a standard way to operate under identity, authority, work state, doctrine, verification, replay, and mutation semantics.

A product-neutral object model would make those semantics explicit rather than binding them to one implementation:

Product-neutral means technology-independent: the object model is not tied to this prototype, any issue tracker, a trading workflow, or MCP. It is a vocabulary for institutional control objects and lifecycle rules that another implementation could store in a database, workflow engine, issue tracker, or tool server while preserving the same authority semantics.

Object	Key fields	Lifecycle	Why tool protocols alone do not cover it
Organization	owner boundary, member principals, role families, work surfaces, doctrine surfaces, verifier topology	created, configured, staffed, mutated, audited	tool protocols expose tools and resources, not institutional membership, authority, finality, or doctrine ownership
Employee principal	identity, role, tools, inbox, status, history	hired, active, updated, suspended, terminated	tool protocols do not define persistent labor identity
Role contract	owned decisions, forbidden decisions, outputs, escalation, handoff interface	created, patched, versioned, retired	tool access is not authority
Playbook doctrine	scope, inputs, outputs, gates, no-action criteria, version	created, read, patched, versioned	retrieved documentation is not durable operating doctrine
Governing work record	owner, state, evidence, blocker, next action, verifier state	opened, routed, blocked, closed, replayed	tool protocols do not define case truth or closure
Work message	sender, recipient, work item, ask, reply flag, inbox state	sent, actionable, superseded, already done	chat is not work-state transfer
Verifier gate	artifact status, criteria, decision, execution authorization, next owner, authority effect	accept, repair, reroute, decline, needs review	validation alone is not institutional authority
No-action state	type, explicit zero, evidence basis, authority basis, next owner/action, replay requirement	proposed, verified, closed, replayed, classified case-only	capability protocols do not define when not acting is valid work
Closure decision	accepting authority, final state, dependency, rejection/reroute reason, replay obligation	proposed, accepted, kept open, rejected, rerouted, closed	execution protocols do not define institutional finality
Replay record	original path, outcome truth, role/playbook versions, learning destination	pending, completed, mutated, classified case-only	traces do not decide doctrine change
Institutional mutation	trigger, source evidence, destination layer, authorized actor, mutation type, activation step, expected behavior change, recurrence check	proposed, authorized, applied, activated, rechecked	administrative APIs do not define governed adaptation
Doctrine patch	target artifact, old string, new string, reason, authority, version result	proposed, applied, rejected, versioned	text editing is not enough for auditable institutional mutation

For example, in a synthetic security-incident desk, an `employee principal` can be a persistent triage analyst, a `role contract` can permit alert classification while forbidding containment approval, a `governing work record` can own incident truth, a `verifier gate` can accept a packet as reconstructible while declining containment authorization, a `no-action state` can record monitor-only handling, a `closure decision` can close the triage phase while replay stays open, and an `institutional mutation` can update a playbook or role boundary

after repeated evidence. The public synthetic example keeps this walkthrough at the organization-protocol level rather than giving security-operations instructions.

9.7 Why The Prediction-Market Desk Is A Useful Critical Case

Prediction-market trading is a strong empirical stress test because it combines uncertainty, external evidence, delayed outcomes, and real action. The domain punishes unsupported action and makes no-action meaningful.

The general mechanisms transfer to other domains with:

- staged technical work;
- external evidence;
- role authority;
- review and acceptance criteria;
- delayed outcome truth;
- operational consequences;
- repeated workflow learning.

Examples include software maintenance, security operations, research operations, legal and compliance workflows, healthcare administration, financial research, and supply-chain incident response.

10. Ethics, Risk, And Compliance

This deployment operated in a real-money prediction-market setting. The paper therefore avoids presenting the work as trading advice, strategy disclosure, or evidence that AI systems may act without institutional controls.

This paper is not financial advice, not a trading strategy, and not a recommendation to deploy autonomous trading agents.

The paper intentionally omits market names, order sizes, account details, strategy thresholds, and timing details that could expose operational strategy or encourage imitation.

The relevant risk controls are:

- human owner boundary;
- role-based execution authority;
- verifier gates before action;
- no-action states;
- risk and exposure constraints;
- manager closure;
- replay and auditability;
- separation between live closure and delayed outcome truth;
- privacy-preserving evidence extraction.

The ethical claim is not that autonomy is inherently desirable. It is that if AI systems are used for high-skill labor, the responsible path is to make authority, evidence, review, stopping, and learning explicit.

11. Limitations And Threats To Validity

11.1 Internal Validity

The organization changed while it operated. This is realistic for applied deployment but limits clean causal inference. Interventions overlap, and this study does not estimate exact before/after windows. The system owner also shaped the institution, selected the research framing, and directed parts of the evidence extraction process. This is normal for an applied field study, but it means the paper should avoid claiming neutral observation from outside the system.

11.2 External Validity

The empirical setting is one AI organization in one high-skill domain. The paper can support depth and mechanism discovery, not general causal certainty. Prediction markets provide unusually clear settlement and

outcome truth; other domains may have slower, noisier, or more contested feedback. Single-manager finality also prevents split-brain mutation but may limit scale. Subsequent work could explore delegated managers, domain managers, scoped mutation authority, or quorum review.

11.3 Measurement Validity

The evidence emphasizes high-signal cases plus selected bounded windows. This study does not estimate the broader denominators required for organization-wide rate claims. Roles and playbooks remain natural-language artifacts; explicitness helps but does not guarantee compliance. Future systems may need machine-checkable doctrine or typed organizational policies.

Duplicate/canonical-parent repair is source-backed and supports the workboard graph-truth mechanism, but the available extraction is not sufficient for a bounded quantitative workboard-frontier result.

11.4 Coding Reliability

Evidence extraction and coding were author-directed. Subsequent work could use independent coders or blinded review for a subset of work records, messages, verifier decisions, no-action states, and replay records.

11.5 Human-Owner Boundary

The organization was AI-staffed in daily labor, but the human owner remained an authority boundary. The paper should not claim fully unsupervised human-free operation. The observed system is best understood as a human-owned, AI-staffed institution.

11.6 Reproducibility

Privacy and operational sensitivity limit how much raw evidence can be published. Public artifacts should use sanitized field sets, aggregate metrics, and publication-ready case labels.

The public package lets readers inspect and try the institutional-control semantics without exposing private trading details. It includes the technical report with a product-neutral organization-protocol object model, API/MCP documentation for sandboxed use, and a synthetic non-trading workboard example. The report itself includes paper-safe artifact field sets for role contracts, work records, messages, verifier decisions, closure decisions, and replay records. These field sets are distilled from evolved institutional artifacts, not presented as fixed external templates. In the deployment, such forms matured through repeated institutional use, failure pressure, verifier feedback, and doctrine mutation. The public API demonstrates institutional semantics, not autonomous trading.

The synthetic example is a demonstration of organization-protocol semantics, not empirical evidence and not a benchmark.

12. Conclusion

The central lesson of this deployment is that agent capability is not the same as auditable labor control. This study suggests that high-skill AI work may benefit from durable institutional control: employees, authority, work records, messages, doctrine, evidence gates, no-action states, replay, and governed adaptation.

The AI-staffed prediction-market desk began as a practical experiment in AI labor and became evidence for a broader claim. Many failures were not addressed by asking agents to think harder. They were addressed by changing the organization around the agents.

The next frontier is not merely agents that can act. It is organizations that can make AI action accountable.

13. Figures, Tables, And Appendices

13.1 Included Figures

Main-body figures included in this report:

1. **Figure 1: From Models To AI Organizations.**
2. **Figure 2: Institutional Control Stack.**
3. **Figure 3: Workboard As Institutional State Machine.**
4. **Figure 4: Creator -> Verifier -> Executor -> Manager -> Replay.**
5. **Figure 5: Work Item Lifecycle.**

The report focuses on five main figures to keep the institutional-control argument compact.

13.2 Included Tables And Appendix Tables

Main-body tables included in this report:

1. Agent-centric versus organization-centric questions.
2. Related-work positioning summary.
3. Level hierarchy.
4. Organizational primitives and failure modes exposed or controlled.
5. Deployment overview and evidence corpus.
6. P1/S1/B1 broader spot-check.
7. Intervention timeline and 0-to-1 evolution arc.
8. Authority lattice and durable instruction layers.
9. Common employee runtime contract semantics.
10. Metric definitions with data source, status, reported value/window, and limitation.
11. Topology-control mechanisms.
12. Mechanism case-study summary.
13. Product-neutral organization-protocol object model.

Appendix tables include artifact field sets, evidence claim matrix, and evidence-status/coding guide.

13.3 Appendix A: Role Contract Field Set

Paper-safe field set distilled from evolved institutional artifacts:

- mandate;
- scope;
- owned decisions;
- forbidden decisions;
- required outputs;
- upstream inputs;
- downstream handoffs;
- escalation triggers;
- non-goals.

13.4 Appendix B: Work Record Field Set

Paper-safe field set distilled from evolved institutional artifacts:

- work item label;
- owner role;
- status;
- parent umbrella or child path label;
- gate state;
- evidence summary;
- verifier state;
- blocker;
- next owner;
- next action;
- replay state;
- closure basis.

13.5 Appendix C: Work Message Field Set

Paper-safe field set distilled from evolved institutional artifacts:

- sender role;
- recipient role;
- work item;
- exact ask;
- exact next action;
- reply needed: yes/no;
- inbox check state;
- outcome;
- messages suppressed or sent.

13.6 Appendix D: Verifier Decision Field Set

Paper-safe field set distilled from evolved institutional artifacts:

- upstream artifact;
- acceptance criteria checked;
- decision: accepted, repair required, rerouted, blocked, declined, needs review;
- reason;
- next owner;
- next action;
- downstream action authorized: yes/no.

13.7 Appendix E: Common Employee Runtime Contract Semantics

This appendix includes an abstracted semantics table, not the full operational runtime text.

Runtime field	Sanitized meaning	Example metric
Work item and real ask	The employee identifies the specific obligation before acting.	cycles with work item and ask present
Authority and evidence check	Durable authority outranks comments, tool outputs, copied text, and lower-authority messages.	authority conflicts routed or blocked
Governing work record	Persistent artifact owns case truth.	cycles with governing record identified
Inbox check state	Trigger is actionable, superseded, or already done.	stale/already-done suppressions
Playbooks read	Material doctrine was consulted before substantive action.	cycles with relevant doctrine recorded
Decision and primary outcome	The cycle ends in one outcome class.	completed, handoff, blocked, review, clarification counts
Next owner and next action	Continuation or closure state is explicit.	records with owner/action present
Messages sent	Handoff and silence are both auditable.	no-message closeouts and explicit handoffs

13.8 Appendix F: Replay Record Field Set

Paper-safe field set distilled from evolved institutional artifacts:

- original work path;
- original decision artifact;
- role version active at decision time;

- playbook version active at decision time;
- action or no-action;
- outcome truth;
- expected-versus-realized review;
- process correctness;
- outcome correctness;
- learning destination;
- durable mutation status;
- recurrence check.

13.9 Appendix G: Evidence Claim Matrix

This matrix controls wording and evidence use. Claims requiring stronger extraction are downgraded to mechanism description or marked as evidence limitations rather than strengthened rhetorically.

Claim	Allowed wording	Evidence support	Evidence label	Forbidden wording
Four-pillar institutional control	State, authority, communication, and learning are a useful organizing frame for the observed mechanisms.	Framework synthesis plus Results organization.	Conceptual claim	These four pillars are complete, universal, or proven sufficient.
State control	Made work truth more auditable in S1.	S1 closeout records and fixed stamps.	Bounded metric claim	Established a global reliability rate.
Single-step scope control	Each employee cycle is bounded to one authorized outcome, reducing scope ambiguity and making audit easier.	Harness evidence and S1 final closeout records.	Mechanism evidence claim	Proved over-expansion reduction, reliability improvement, or complete scope control.
Authority control	Turned review into state decisions in observed verifier paths.	S1 verifier comments and reroutes.	Bounded metric claim	Established complete autonomous execution control.
Action-gate decline	Verifier topology separates artifact review from execution authorization in observed verifier paths.	Verifier comments and no-execution dispositions.	Mechanism evidence claim	Verifier decline proves safety or improved decision quality.
Workboard state machine	Compiled owner intent into live/retro umbrellas, child paths, activations, gates, and replay obligations.	S1/R1/V1 sprint structure and workboard evolution.	Mechanism evidence claim	Proved the topology is optimal or universally required.

Claim	Allowed wording	Evidence support	Evidence label	Forbidden wording
Durable artifact growth	Repeated operational failures were promoted into role, work-record, message, verifier, replay, tool, or playbook artifact forms under manager/owner authority.	Intervention timeline, mechanism cases A-J, evolved artifact field sets, and runtime/tool-surface evidence.	Mechanism evidence claim	The system autonomously bootstrapped itself, proved self-improvement, or reduced recurrence.
False-motion control	Made stale, duplicate, and non-material work suppressible in observed packets and closeouts.	M1 packets and S1 closeouts.	Mechanism evidence claim	Showed there are no remaining loops.
Learning control	Separated stopping, replay, and doctrine-change destinations.	S1 replay records and retro closeout.	Bounded metric claim	Established recurrence reduction.
Explicit zero	Explicit zero is a verified no-action state where the organization records that no live path or executable action remains.	No-action mechanism evidence and workboard state evidence.	Mechanism evidence claim	Explicit zero proves correctness or successful outcome.
Topology control	Made routing and authority more explicit through role families, lanes, default chains, manager finality, and synchronized truth surfaces.	Organization evolution and mechanism cases.	Mechanism evidence claim	Established a general best organizational topology.
Person-role-aware tool surface	Shows tool capability interpreted through employee principal, role, work state, and manager/user authority.	Tool-surface evolution and authority-control cases.	Mechanism evidence claim	Claimed tool protocols alone enforce all institutional authority.
Institutional control	Suggested as useful for high-skill AI labor.	Field study plus mechanism evidence.	Conceptual claim	Claimed this is mandatory for all reliable AI labor.
Durable mutation	Describes a mechanism for changing doctrine, roles, tools, staffing, topology, or message protocol.	Selected evolution/tool records.	Mechanism evidence claim	Reports aggregate mutation rates.

Claim	Allowed wording	Evidence support	Evidence label	Forbidden wording
Authorized institutional mutation / tool-parity-gated activation	Shows one source-backed activation-gating mechanism where rollout waited until required role, tool, routing, and activation surfaces were directly evidenced as aligned.	Case I, tool-parity-gated activation.	Mechanism evidence claim	Recurrence reduction, causal reliability improvement, complete activation safety, safe autonomous rollout, successful rollout.
Targeted doctrine patching	Identifies anchored edits as a mechanism for local doctrine change.	Source-backed tool behavior and sanitized mechanism example.	Mechanism evidence claim; bounded quantitative effects not estimated	Quantifies context-cost savings or error reduction.
Reliability	Made reliability-relevant failures more visible and controllable.	State, authority, communication, and learning evidence.	Design implication	Established global decision-quality improvement.

13.10 Appendix H: Evidence Status And Coding Guide

Current evidence is strongest for mechanism tracing and bounded mature-state audits. It is weaker for aggregate rates.

Evidence type	Current status	Use in this report
Mechanism case packets from work records	extracted	main qualitative results
Message evidence packets	extracted	message and verifier cases
S1 direct activation count	extracted mechanism evidence	10 direct activation messages for 20 child records from S1 live umbrella; not a full inbox census
Single-step scope control	mechanism evidence extracted	runtime-contract rule plus S1 final closeout records; no over-expansion reduction rate
Intervention timeline	extracted/inferred mix	empirical backbone
Harness evolution	inferred from mature runtime contract plus linked mechanism evidence	method claim with caveat; historical prompt-version sequence was not fully reconstructed
Metric definitions	defined	evaluation framework
Denominator-bounded metric values	extracted for selected sprint windows	sample-labeled Results counts
Broader P1/B1 spot-check	shallow metadata coding	selection-bias context, not full-corpus rates
Case I, tool-parity-gated activation	extracted mechanism evidence	activation-gating mechanism; no recurrence, reliability, or safety rate
Case J, durable artifact growth	mechanism evidence extracted	repeated failures promoted into durable artifact forms under manager/owner authority; no self-improvement, recurrence, or generalization rate

Evidence type	Current status	Use in this report
Tool-parity metrics	source artifacts identified; bounded estimates not reported	bounded counts are not reported for this mechanism; no denominator or time-to-parity value reported
Recurrence windows	partial	this study does not estimate recurrence after mutation
Targeted doctrine patch quantitative evidence	defined but not estimated in this study	method claim with explicit caveat

Harness evolution is inferred from the mature runtime contract and linked mechanism evidence; the historical prompt-version sequence was not fully reconstructed.

For metrics defined but not estimated in this study, the coding schema is:

- current owner present: yes/no;
- next action present: yes/no;
- role authority respected: yes/no/unclear;
- required evidence present: yes/no/partial;
- verifier decision type: accept/repair/reroute/decline/block/needs review;
- closure validity: valid/invalid/open/unclear;
- no-action type: none/no-send/decline/stand-down/blocked/already-done/explicit zero;
- replay state: not required/pending/complete/missing;
- mutation destination: role/playbook/tool/staffing/topology/workboard/message/case-only/none;
- recurrence: none observed/observed/not enough exposure.

References

Agent Control Protocol. Architecture, 2026. URL https://agentcontrolprotocol.xyz/. Accessed: 2026-05-03.

Chris Argyris and Donald A. Schön. *Organizational Learning: A Theory of Action Perspective*. Addison-Wesley, Reading, MA, 1978.

Guido Boella, Leendert van der Torre, and Harko Verhagen. Introduction to normative multiagent systems. *Computational and Mathematical Organization Theory*, 12:71–79, 2006. doi: 10.1007/s10588-006-9537-7. URL https://doi.org/10.1007/s10588-006-9537-7.

CrewAI. CrewAI documentation, 2026. URL https://docs.crewai.com/. Accessed: 2026-05-03.

Marlon Dumas, Marcello La Rosa, Jan Mendling, and Hajo A. Reijers. *Fundamentals of Business Process Management*. Springer Berlin Heidelberg, Berlin, Heidelberg, 2 edition, 2018. ISBN 9783662565094. doi: 10.1007/978-3-662-56509-4. URL https://doi.org/10.1007/978-3-662-56509-4.

Marc Esteva, David de la Cruz, and Carles Sierra. Engineering open multi-agent systems as electronic institutions. In *Proceedings of the Nineteenth National Conference on Artificial Intelligence*. AAAI Press, 2004. URL https://s.aaai.org/Library/AAAI/2004/aaai04-153.php.

Marcelo Fernandez. Agent control protocol: Admission control for agent actions, 2026. URL https://arxiv.org/abs/2603.18829.

Sirui Hong, Mingchen Zhuge, Jiaqi Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework, 2024. URL https://arxiv.org/abs/2308.00352.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues?, 2024. URL https://arxiv.org/abs/2310.06770.

LangChain. LangGraph durable execution, 2026. URL https://docs.langchain.com/oss/python/langgraph/durable-execution. Accessed: 2026-05-03.

Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. AgentBench: Evaluating LLMs as agents. In *International Conference on Learning Representations*, 2024. URL https://arxiv.org/abs/2308.03688. Latest arXiv revision 2025.

Model Context Protocol. Basic specification, version 2025-11-25, 2025a. URL https://modelcontextprotocol.io/specification/2025-11-25/basic. Accessed: 2026-05-03.

Model Context Protocol. Specification, version 2025-11-25, 2025b. URL https://modelcontextprotocol.io/specification/2025-11-25. Accessed: 2026-05-03.

OpenAI. The next evolution of the Agents SDK, 2026a. URL https://openai.com/index/the-next-evolution-of-the-agents-sdk/. Accessed: 2026-05-03.

OpenAI. OpenAI Agents SDK documentation: Guardrails, 2026b. URL https://openai.github.io/openai-agents-python/guardrails/. Accessed: 2026-05-03.

OpenAI. OpenAI Agents SDK documentation: Tracing, 2026c. URL https://openai.github.io/openai-agents-python/tracing/. Accessed: 2026-05-03.

Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior, 2023. URL https://arxiv.org/abs/2304.03442.

Federico Pierucci, Marcello Galisai, Marcantonio Syrnikov Bracale, Matteo Prandi, Piercosma Bisconti, Francesco Giarrusso, Olga Sorokoletova, Vincenzo Suriani, and Daniele Nardi. Institutional AI: A governance framework for distributional AGI safety, 2026. URL https://arxiv.org/abs/2601.10599.

Vasundra Srinivasan. Bridging protocol and production: Design patterns for deploying AI agents with model context protocol, 2026. URL https://arxiv.org/abs/2603.13417. Preprint.

Marcantonio Bracale Syrnikov, Federico Pierucci, Marcello Galisai, Matteo Prandi, Piercosma Bisconti, Francesco Giarrusso, Olga Sorokoletova, Vincenzo Suriani, and Daniele Nardi. Institutional AI: Governing LLM collusion in multi-agent Cournot markets via public governance graphs, 2026. URL https://arxiv.org/abs/2601.11369.

Wil van der Aalst. *Process Mining: Data Science in Action*. Springer Berlin Heidelberg, Berlin, Heidelberg, 2 edition, 2016. ISBN 9783662498514. doi: 10.1007/978-3-662-49851-4. URL https://doi.org/10.1007/978-3-662-49851-4.

Karl E. Weick and Kathleen M. Sutcliffe. *Managing the Unexpected: Sustained Performance in a Complex World*. John Wiley & Sons, Hoboken, NJ, 3 edition, 2015. ISBN 9781118862414.

Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation, 2023. URL https://arxiv.org/abs/2308.08155.

Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments, 2024. URL https://arxiv.org/abs/2404.07972.

Frank F. Xu, Yufan Song, Boxuan Li, Yuxuan Tang, Kritanjali Jain, Mengxue Bao, Zora Z. Wang, Xuhui Zhou, Zhitong Guo, Murong Cao, Mingyang Yang, Hao Yang Lu, Amaad Martin, Zhe Su, Leander Maben, Raj Mehta, Wayne Chi, Lawrence Jang, Yiqing Xie, Shuyan Zhou, and Graham Neubig. TheAgentCompany: Benchmarking LLM agents on consequential real world tasks, 2024. URL https://arxiv.org/abs/2412.14161. Preprint; latest arXiv revision 2025.

Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains, 2024. URL https://arxiv.org/abs/2406.12045.

Asaf Yehudai, Lilach Eden, Alan Li, Guy Uziel, Yilun Zhao, Roy Bar-Haim, Arman Cohan, and Michal Shmueli-Scheuer. Survey on evaluation of LLM-based agents, 2026. URL https://arxiv.org/abs/2503.16416. Submitted 2025; revised 2026.

Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. WebArena: A realistic web environment for building autonomous agents, 2024. URL https://arxiv.org/abs/2307.13854.